Appendix: Complete R Code for FINSTAD Case 1

CRUZ, Ricardo Miguel Iñigo GALEDO, Enrique Lorenzo Hermoso
SEBALLOS, Josiah Dweyn Panganiban SEECHUNG, Camille Castro
SISON, Aaron Joshua Estacio

Table of contents

Appendix A1 - Setup & Library Import

Loads the required R libraries (dplyr, tidyr, lubridate, ggplot2, etc.) and defines global helper functions used throughout the analysis. The working directory is resolved automatically to ensure data file paths work regardless of whether the script runs from the project root or the scripts/ folder.

```
# Load necessary libraries
library(dplyr)
library(tidyr)
library(lubridate)
library(DBI)
library(RSQLite)
library(ggplot2)
library(scales)
```

```r
library(kableExtra)

# Safe table formatter for PDF to prevent overlapping columns and
 ↪  overflowing margins
safe_kable <- function(data, caption = NULL, digits = 4) {
  knitr::kable(data, caption = caption, digits = digits, format = "latex",
 ↪  booktabs = TRUE) %>%
    kable_styling(font_size = 8, latex_options = c("scale_down",
     ↪  "HOLD_position")) %>%
    column_spec(1:ncol(data), width_min = "1.5cm")
}

# Set working directory to project root so data paths resolve consistently
# Ensure data paths resolve correctly regardless of working directory
if (dir.exists("data")) {
  data_dir <- "data"
} else if (dir.exists("../data")) {
  data_dir <- "../data"
} else {
  stop("Cannot find data/ directory. Run from project root or scripts/.")
}

# Global options
options(stringsAsFactors = FALSE, scipen = 999, width = 72)

# Helper to parse BDO volume strings (e.g. "2.50M" to 2500000)
parse_bdo_volume <- function(x) {
  x <- gsub(",", "", x)
  mult <- ifelse(grepl("M", x, ignore.case = TRUE), 1e6,
                 ifelse(grepl("K", x, ignore.case = TRUE), 1e3, 1))
  x_num <- as.numeric(gsub("[A-Za-z%]", "", x))
  x_num * mult
}
```

Appendix A2 - Ticker Definitions

Defines the asset ticker symbols used throughout the case: AAPL (US equity), BDO.PS (Philippine bank stock), and BTC-USD (cryptocurrency).

```r
# Define tickers
us_asset <- "AAPL"
ph_asset <- "BDO.PS"
crypto <- "BTC-USD"
```

Appendix A3 - Data Loading

Author: GALEDO, Enrique Lorenzo Hermoso Loads all 8 CSV datasets from the data/ directory into R data frames. Records the original observation count for each dataset before any cleaning operations. Datasets include 4 asset price series (AAPL, BDO, BTC, SPY) and 4 FRED macroeconomic series (CPI, DGS10, FEDFUNDS, VIX). The raw CSVs contain observations well beyond the 2020-2025 analysis window; filtering to the sample period is applied in A4 across all series simultaneously.

```
# Data Loading -- GALEDO, Enrique Lorenzo Hermoso (Sec 3: Data Acquisition)

# Load all 8 CSVs from the data directory
aapl <- read.csv(file.path(data_dir, "AAPL.csv"), stringsAsFactors = FALSE)
bdo  <- read.csv(file.path(data_dir, "BDO.csv"), stringsAsFactors = FALSE,
 ↪  fileEncoding = "UTF-8-BOM")
btc  <- read.csv(file.path(data_dir, "BTC.csv"), stringsAsFactors = FALSE)
spy  <- read.csv(file.path(data_dir, "SPY.csv"), stringsAsFactors = FALSE)
cpi     <- read.csv(file.path(data_dir, "CPI.csv"), stringsAsFactors =
 ↪  FALSE)
dgs10   <- read.csv(file.path(data_dir, "DGS10.csv"), stringsAsFactors =
 ↪  FALSE)
fedfunds <- read.csv(file.path(data_dir, "FEDFUNDS.csv"), stringsAsFactors =
 ↪  FALSE)
vix     <- read.csv(file.path(data_dir, "VIX.csv"), stringsAsFactors =
 ↪  FALSE)

# Record original observations before any processing
obs_original <- data.frame(
  Dataset = c("AAPL", "BDO", "BTC", "SPY", "CPI", "DGS10", "FEDFUNDS",
   ↪  "VIX"),
  Original_Obs = c(nrow(aapl), nrow(bdo), nrow(btc), nrow(spy),
                   nrow(cpi), nrow(dgs10), nrow(fedfunds), nrow(vix))
)

safe_kable(obs_original, caption="Original Observations per Dataset")
```

Table 1: Original Observations per Dataset

Dataset	Original_Obs
AAPL	2512
BDO	2441
BTC	3652
SPY	2514
CPI	952
DGS10	16105
FEDFUNDS	863
VIX	9216

Appendix A4 - Data Cleaning

Author: SISON, Aaron Joshua Estacio Standardises date formats across all 8 datasets, extracts December 2019 seed values for monthly macro series, filters to the 2020-2025 sample period, generates a canonical Mon-Fri business calendar, left-joins each asset onto the calendar, recalculates BTC returns after weekend removal, forward-fills Close prices and zero-fills Daily_Return on non-trading days, seeds CPI and FEDFUNDS with December 2019 values to avoid cold-start nulls, and forward-fills all four macroeconomic variables via LOCF.

```
# Sec 4: Data Cleaning -- SISON, Aaron Joshua Estacio

# Standardise date formats
# AAPL, BTC, SPY: POSIXct-style strings like "2016-06-30 00:00:00-04:00"
aapl <- aapl |> mutate(Date = as.Date(ymd_hms(Date)))
btc  <- btc  |> mutate(Date = as.Date(ymd_hms(Date)))
spy  <- spy  |> mutate(Date = as.Date(ymd_hms(Date)))

# BDO: MM/DD/YYYY
bdo <- bdo |> mutate(Date = mdy(Date))

# CPI, DGS10, FEDFUNDS, VIX: YYYY-MM-DD
cpi      <- cpi      |> mutate(Date = ymd(Date))
dgs10    <- dgs10    |> mutate(Date = ymd(Date))
fedfunds <- fedfunds |> mutate(Date = ymd(Date))
vix      <- vix      |> mutate(Date = ymd(Date))

# Extract December 2019 seed values for CPI and FEDFUNDS
# These provide the initial carry-forward for the first business days
# of January 2020 before the first 2020 monthly release
cpi_dec2019_val <- cpi |> filter(Date == as.Date("2019-12-01")) |>
 ↪  pull(Value)
fedfunds_dec2019_val <- fedfunds |> filter(Date == as.Date("2019-12-01")) |>
 ↪  pull(Value)

# Save pre-filter macro data for CPI_YoY computation (needs 12-month lag)
cpi_pre_filter <- cpi
dgs10_pre_filter <- dgs10
fedfunds_pre_filter <- fedfunds
vix_pre_filter <- vix

# Filter ALL datasets to 2020-01-01 to 2025-12-31
start_date <- as.Date("2020-01-01")
end_date   <- as.Date("2025-12-31")

aapl <- aapl |> filter(Date >= start_date & Date <= end_date)
bdo  <- bdo  |> filter(Date >= start_date & Date <= end_date)
```

```r
btc  <- btc  |> filter(Date >= start_date & Date <= end_date)
spy  <- spy  |> filter(Date >= start_date & Date <= end_date)
cpi     <- cpi     |> filter(Date >= start_date & Date <= end_date)
dgs10   <- dgs10   |> filter(Date >= start_date & Date <= end_date)
fedfunds <- fedfunds |> filter(Date >= start_date & Date <= end_date)
vix     <- vix     |> filter(Date >= start_date & Date <= end_date)

# Observations after date filtering (before calendar join)
obs_date_filtered <- data.frame(
  Dataset = c("AAPL", "BDO", "BTC", "SPY", "CPI", "DGS10", "FEDFUNDS",
   ↪  "VIX"),
  After_Date_Filter = c(nrow(aapl), nrow(bdo), nrow(btc), nrow(spy),
                        nrow(cpi), nrow(dgs10), nrow(fedfunds), nrow(vix))
)

# Generate canonical Mon-Fri business calendar
# No exchange-specific holiday filtering; holidays are bridged via
 ↪  forward-fill
canonical_calendar <- data.frame(
  Date = seq.Date(from = start_date, to = end_date, by = "day")
) |>
  filter(!weekdays(Date) %in% c("Saturday", "Sunday"))

# Left-join each asset onto the calendar
# Assets with no observation on a given calendar date get NA
 ↪  Close/Daily_Return
# BTC loses approximately 626 weekend rows (2,192 to 1,566)
aapl <- canonical_calendar |> left_join(aapl, by = "Date") |> arrange(Date)
bdo  <- canonical_calendar |> left_join(bdo, by = "Date")  |> arrange(Date)
btc  <- canonical_calendar |> left_join(btc, by = "Date")  |> arrange(Date)
spy  <- canonical_calendar |> left_join(spy, by = "Date")  |> arrange(Date)

# Recalculate daily returns on the filtered business-day grid
# After weekend removal, Monday's return uses Friday's close, not Sunday's
# This must happen AFTER the left-join but BEFORE forward-fill
aapl <- aapl |> arrange(Date) |> mutate(Daily_Return = (Close / lag(Close))
 ↪  - 1)
bdo  <- bdo  |> arrange(Date) |> mutate(Daily_Return = (Price / lag(Price))
 ↪  - 1)
btc <- btc |> arrange(Date) |> mutate(Daily_Return = (Close / lag(Close)) -
 ↪  1)
spy  <- spy  |> arrange(Date) |> mutate(Daily_Return = (Close / lag(Close))
 ↪  - 1)
```

```r
# Parse BDO volume strings before column rename
if ("Vol." %in% colnames(bdo)) {
  bdo <- bdo |> mutate(`Vol.` = parse_bdo_volume(`Vol.`))
}

# Rename BDO columns early for forward-fill consistency
bdo <- bdo |> rename(Close = Price, Volume = `Vol.`)

# Seed first-row NAs on 2020-01-01 (New Year's Day holiday, no trading)
aapl_first_close <- aapl |> filter(!is.na(Close)) |> pull(Close) |> head(1)
bdo_first_close <- bdo |> filter(!is.na(Close)) |> pull(Close) |> head(1)
spy_first_close <- spy |> filter(!is.na(Close)) |> pull(Close) |> head(1)
if (is.na(aapl$Close[1])) { aapl$Close[1] <- aapl_first_close }
if (is.na(bdo$Close[1])) { bdo$Close[1] <- bdo_first_close }
if (is.na(spy$Close[1])) { spy$Close[1] <- spy_first_close }

# Forward-fill asset Close prices and zero-fill Daily_Return
# Close: last executable price carried forward (no trade, no price change)
# Daily_Return: zero on non-trading days (no trade, no return)
aapl <- aapl |> tidyr::fill(Close, .direction = "down") |>
  mutate(Daily_Return = if_else(is.na(Daily_Return), 0, Daily_Return))
bdo  <- bdo  |> tidyr::fill(Close, .direction = "down") |>
  mutate(Daily_Return = if_else(is.na(Daily_Return), 0, Daily_Return))
spy  <- spy  |> tidyr::fill(Close, .direction = "down") |>
  mutate(Daily_Return = if_else(is.na(Daily_Return), 0, Daily_Return))
btc  <- btc  |> mutate(Daily_Return = if_else(is.na(Daily_Return), 0,
 ↪  Daily_Return))

# Rename macro variable Value columns to meaningful names
cpi_clean     <- cpi     |> rename(CPI = Value)
dgs10_clean   <- dgs10   |> rename(DGS10 = Value)
fedfunds_clean <- fedfunds |> rename(FEDFUNDS = Value)
vix_clean     <- vix     |> rename(VIX = Value)

# Left-join each macro variable onto the calendar and forward-fill
# CPI: monthly series; seed first row with December 2019 value
cpi <- canonical_calendar |>
  left_join(cpi_clean, by = "Date") |>
  arrange(Date)
cpi_nulls_before <- sum(is.na(cpi$CPI))
if (is.na(cpi$CPI[1])) { cpi$CPI[1] <- cpi_dec2019_val }
cpi <- cpi |> tidyr::fill(CPI, .direction = "down")

# DGS10: business-daily series
# After left-join, gaps exist where FRED did not report
```

```r
# on days corresponding to non-trading dates in the calendar
dgs10 <- canonical_calendar |>
  left_join(dgs10_clean, by = "Date") |>
  arrange(Date)
dgs10_nulls_before <- sum(is.na(dgs10$DGS10))
dgs10_first <- dgs10 |> filter(!is.na(DGS10)) |> pull(DGS10) |> head(1)
if (is.na(dgs10$DGS10[1])) { dgs10$DGS10[1] <- dgs10_first }
dgs10 <- dgs10 |> tidyr::fill(DGS10, .direction = "down")

# FEDFUNDS: meeting-based series; seed first row with December 2019 value
fedfunds <- canonical_calendar |>
  left_join(fedfunds_clean, by = "Date") |>
  arrange(Date)
fedfunds_nulls_before <- sum(is.na(fedfunds$FEDFUNDS))
if (is.na(fedfunds$FEDFUNDS[1])) { fedfunds$FEDFUNDS[1] <-
 ↪  fedfunds_dec2019_val }
fedfunds <- fedfunds |> tidyr::fill(FEDFUNDS, .direction = "down")

# VIX: business-daily series
vix <- canonical_calendar |>
  left_join(vix_clean, by = "Date") |>
  arrange(Date)
vix_nulls_before <- sum(is.na(vix$VIX))
vix_first <- vix |> filter(!is.na(VIX)) |> pull(VIX) |> head(1)
if (is.na(vix$VIX[1])) { vix$VIX[1] <- vix_first }
vix <- vix |> tidyr::fill(VIX, .direction = "down")

# Observations after calendar join and forward-fill
obs_after_ffill <- data.frame(
  Dataset = c("AAPL", "BDO", "BTC", "SPY", "CPI", "DGS10", "FEDFUNDS",
   ↪  "VIX"),
  After_Calendar = c(nrow(aapl), nrow(bdo), nrow(btc), nrow(spy),
                     nrow(cpi), nrow(dgs10), nrow(fedfunds), nrow(vix)),
  Nulls_Remaining = c(sum(is.na(aapl)), sum(is.na(bdo)), sum(is.na(btc)),
                      sum(is.na(spy)), sum(is.na(cpi)), sum(is.na(dgs10)),
                      sum(is.na(fedfunds)), sum(is.na(vix)))
)

# Forward-fill summary: null counts per macro column before and after
ffill_macro <- data.frame(
  Variable = c("CPI", "DGS10", "FEDFUNDS", "VIX"),
  Nulls_Before_Fill = c(cpi_nulls_before, dgs10_nulls_before,
                        fedfunds_nulls_before, vix_nulls_before),
  Nulls_After_Fill = c(0, 0, 0, 0)
)
```

```
# Display tables
safe_kable(obs_date_filtered, caption="Observations After Date Filtering
 ↪  (Before Calendar Join)")
```

Table 2: Observations After Date Filtering (Before Calendar Join)

Dataset	After_Date_Filter
AAPL	1508
BDO	1468
BTC	2192
SPY	1508
CPI	71
DGS10	1500
FEDFUNDS	72
VIX	1535

```
safe_kable(obs_after_ffill, caption="Observations After Calendar Join and
 ↪  Forward-Fill")
```

Table 3: Observations After Calendar Join and Forward-Fill

Dataset	After_Calendar	Nulls_Remaining
AAPL	1566	232
BDO	1566	490
BTC	1566	0
SPY	1566	290
CPI	1566	0
DGS10	1566	0
FEDFUNDS	1566	0
VIX	1566	0

```
safe_kable(ffill_macro, caption="Forward-Fill Summary: Null Counts Before
 ↪  and After")
```

Table 4: Forward-Fill Summary: Null Counts Before and After

Variable	Nulls_Before_Fill	Nulls_After_Fill
CPI	1515	0
DGS10	66	0
FEDFUNDS	1514	0
VIX	31	0

Appendix A5 - Database Integration

Author: SISON, Aaron Joshua Estacio Demonstrates all four SQL join types (left, inner, full, anti) and justifies the chosen join strategy. Assembles the final analytical dataset by

combining asset data with macroeconomic series. Exports the observation reduction table for inclusion in the main report. All macroeconomic variables are forward-filled in A4, so the join does not introduce new null values.

```
# Sec 5: Database Integration -- SISON, Aaron Joshua Estacio

# Prepare asset data for joining
# BDO columns already renamed in A4 (Price -> Close, Vol. -> Volume)

# Add Asset identifier to each OHLC dataset and select common columns
aapl_clean <- aapl |> mutate(Asset = "AAPL") |>
  select(Date, Asset, Open, High, Low, Close, Volume, Daily_Return)
bdo_clean  <- bdo |> mutate(Asset = "BDO") |>
  select(Date, Asset, Open, High, Low, Close, Volume, Daily_Return)
btc_clean  <- btc |> mutate(Asset = "BTC") |>
  select(Date, Asset, Open, High, Low, Close, Volume, Daily_Return)
spy_clean  <- spy |> mutate(Asset = "SPY") |>
  select(Date, Asset, Open, High, Low, Close, Volume, Daily_Return)

# Combine all assets into one long-format table
assets_all <- bind_rows(aapl_clean, bdo_clean, btc_clean, spy_clean)

# Pivot to wide format: one row per date, columns per asset
assets_wide <- assets_all |>
  select(Date, Asset, Close, Daily_Return) |>
  pivot_wider(names_from = Asset,
              values_from = c(Close, Daily_Return),
              names_sep = "_")

# Macro dataframes from A4 already have renamed columns (CPI, DGS10,
 ↪  FEDFUNDS, VIX)
# and are forward-filled with zero nulls on the canonical calendar

# Demonstration of all four join types

# 1) left_join: keep all asset dates, attach macro data where available
# After forward-fill, left_join produces zero NAs
left_example <- assets_wide |>
  left_join(cpi, by = "Date") |>
  left_join(dgs10, by = "Date") |>
  left_join(fedfunds, by = "Date") |>
  left_join(vix, by = "Date")

# 2) inner_join: only dates present in ALL datasets
# After forward-fill, all series cover all calendar dates
inner_example <- assets_wide |>
```

```r
  inner_join(cpi, by = "Date") |>
  inner_join(dgs10, by = "Date") |>
  inner_join(fedfunds, by = "Date") |>
  inner_join(vix, by = "Date")

# 3) full_join: all dates from any dataset
full_example <- assets_wide |>
  full_join(cpi, by = "Date") |>
  full_join(dgs10, by = "Date") |>
  full_join(fedfunds, by = "Date") |>
  full_join(vix, by = "Date")

# 4) anti_join: asset dates with NO macro data
# After forward-fill, every date has macro data, so anti_join returns 0 rows
anti_example <- assets_wide |>
  anti_join(cpi, by = "Date")

# Build the final analytical dataset
final_dataset <- left_example

# Observation reduction table
# Original_Obs: row count in each raw CSV (before any processing)
# After_Calendar_Filter: rows remaining after date parsing and 2020-2025
 ↪  filter
# After_ForwardFill: rows on the canonical calendar after LOCF

obs_original_vec <- obs_original$Original_Obs
names(obs_original_vec) <- obs_original$Dataset

reduction_table <- data.frame(
  Dataset = c("AAPL", "BDO", "BTC", "SPY", "CPI", "DGS10", "FEDFUNDS",
   ↪  "VIX",
              "Assets (wide)", "Final Dataset"),
  Original_Obs = c(obs_original_vec["AAPL"], obs_original_vec["BDO"],
                   obs_original_vec["BTC"], obs_original_vec["SPY"],
                   obs_original_vec["CPI"], obs_original_vec["DGS10"],
                   obs_original_vec["FEDFUNDS"], obs_original_vec["VIX"],
                   NA, NA),
  After_Calendar_Filter = c(nrow(aapl), nrow(bdo), nrow(btc), nrow(spy),
                            nrow(cpi), nrow(dgs10), nrow(fedfunds),
                             ↪  nrow(vix),
                            nrow(assets_wide), NA),
  After_ForwardFill = c(NA, NA, NA, NA, nrow(cpi), nrow(dgs10),
                        nrow(fedfunds), nrow(vix), NA, nrow(final_dataset))
)
```

```
knitr::kable(reduction_table, caption="Observation Counts Across Processing
 ↪  Stages")
```

Table 5: Observation Counts Across Processing Stages

Dataset	Original_Obs	After_Calendar_Filter	After_ForwardFill
AAPL	2512	1566	NA
BDO	2441	1566	NA
BTC	3652	1566	NA
SPY	2514	1566	NA
CPI	952	1566	1566
DGS10	16105	1566	1566
FEDFUNDS	863	1566	1566
VIX	9216	1566	1566
Assets (wide)	NA	1566	NA
Final Dataset	NA	NA	1566

```
library(knitr)
cat(kable(reduction_table, format = "latex", booktabs = TRUE), file =
 ↪  "../reports/sections/data_cleaning_table.tex")
```

Appendix A6 - Master Dataset Creation & SQL Database Setup

Author: SISON, Aaron Joshua Estacio Builds the master analytical dataset by restructuring into long format (one row per asset-date combination) and saves it as a persistent SQLite database file with human-readable ISO dates. Pre-calculates CPI year-on-year change for inflation analysis. Exports a CSV copy for compatibility with spreadsheet software. All subsequent SQL questions (A7-A16) query this persisted database.

```
# SQLite database setup - creates persisted master dataset

db_path <- "../data/master_dataset.sqlite"
unlink(db_path)
con <- dbConnect(RSQLite::SQLite(), db_path)

# Build long-format analytical dataset for SQL queries
# Each row is one asset-date combination with macro variables attached
sql_long <- bind_rows(
  final_dataset |>
    select(Date, CPI, DGS10, FEDFUNDS, VIX,
```

```
            Close = Close_AAPL, Daily_Return = Daily_Return_AAPL) |>
    mutate(Asset = "AAPL"),
  final_dataset |>
    select(Date, CPI, DGS10, FEDFUNDS, VIX,
           Close = Close_BDO, Daily_Return = Daily_Return_BDO) |>
    mutate(Asset = "BDO"),
  final_dataset |>
    select(Date, CPI, DGS10, FEDFUNDS, VIX,
           Close = Close_BTC, Daily_Return = Daily_Return_BTC) |>
    mutate(Asset = "BTC"),
  final_dataset |>
    select(Date, CPI, DGS10, FEDFUNDS, VIX,
           Close = Close_SPY, Daily_Return = Daily_Return_SPY) |>
    mutate(Asset = "SPY")
)

# Pre-calculate CPI YoY for Q10 (inflation analysis)
# CPI is monthly; YoY change = (current / 12-month-ago) - 1
# CPI_YoY is computed from raw monthly CPI values to preserve genuine
# month-over-12-months change, using pre-filter data so the 12-month
# lag resolves correctly for January 2020
cpi_yoy <- cpi_pre_filter |>
  rename(CPI = Value) |>
  arrange(Date) |>
  mutate(CPI_YoY = (CPI / lag(CPI, 12)) - 1) |>
  filter(Date >= start_date & Date <= end_date) |>
  select(Date, CPI_YoY)

# Join CPI_YoY into the long dataset and forward-fill to cover all trading
 ↪  days
sql_long <- sql_long |>
  left_join(cpi_yoy, by = "Date") |>
  arrange(Date) |>
  tidyr::fill(CPI_YoY, .direction = "down")

# Write to SQLite database
dbWriteTable(con, "integrated_data", sql_long, overwrite = TRUE)
fin_data <- tbl(con, "integrated_data")

# Export CSV copy for spreadsheet compatibility (e.g., Microsoft Excel)
write.csv(sql_long, file.path(data_dir, "master_dataset.csv"),
          row.names = FALSE, na = "")

# Show first 5 rows of the master dataset as a preview
nrow_total <- collect(fin_data) |> nrow()
```

```
safe_kable(head(sql_long, 5), digits=4, caption=paste0("Master Dataset
↪  Preview (First 5 of ", nrow_total, " Rows)"))
```

Table 6: Master Dataset Preview (First 5 of 6264 Rows)

Date	CPI	DGS10	FEDFUNDS	VIX	Close	Daily_Return	Asset	CPI_YoY
2020-01-01	259.127	1.88	1.55	12.47	72.33	0	AAPL	0.026
2020-01-01	259.127	1.88	1.55	12.47	127.35	0	BDO	0.026
2020-01-01	259.127	1.88	1.55	12.47	7200.17	0	BTC	0.026
2020-01-01	259.127	1.88	1.55	12.47	296.13	0	SPY	0.026
2020-01-02	259.127	1.88	1.55	12.47	72.33	0	AAPL	0.026

Appendix A7 - SQL Question 1: Risk-Adjusted Daily Return

Author: SISON, Aaron Joshua Estacio Computes the ratio of mean daily return to standard deviation for each asset, producing a Sharpe-like measure of return efficiency. The corresponding bar chart is displayed in the Data Visualization section.

```
# Q1: Risk-Adjusted Daily Return by Asset -- SISON, Aaron Joshua Estacio

q1 <- fin_data |>
  group_by(Asset) |>
  summarise(
    Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
    SD_Daily_Return = sd(Daily_Return, na.rm = TRUE)
  ) |>
  mutate(Risk_Adjusted_Return = Mean_Daily_Return / SD_Daily_Return) |>
  arrange(desc(Risk_Adjusted_Return)) |>
  collect()

spy_ratio <- q1 |> filter(Asset == "SPY") |> pull(Risk_Adjusted_Return)

safe_kable(q1, digits=4, caption="Q1 Data Output")
```

Table 7: Q1 Data Output

Asset	Mean_Daily_Return	SD_Daily_Return	Risk_Adjusted_Return
AAPL	0.0012	0.0193	0.0627
BTC	0.0023	0.0380	0.0613
SPY	0.0007	0.0127	0.0534
BDO	0.0006	0.0204	0.0310

```
# Write LaTeX table to file for main report inclusion
sink("../reports/sections/q1_table.tex")
cat("\\begin{table}[H]\n\\centering\n")
cat("\\caption{Q1 Data Output}\n")
cat("\\label{tab:q1}\n")
cat("\\begin{tabular}{lrrr}\n")
cat("\\toprule\n")
cat("Asset & Mean\\_Daily\\_Return & SD\\_Daily\\_Return &
 ↪  Risk\\_Adjusted\\_Return \\\\
")
cat("\\midrule\n")
for (i in 1:nrow(q1)) {
  cat(sprintf("%s & %.4f & %.4f & %.4f \\\\
", q1$Asset[i], q1$Mean_Daily_Return[i], q1$SD_Daily_Return[i],
 ↪  q1$Risk_Adjusted_Return[i]))
}
cat("\\bottomrule\n\\end{tabular}\n\\end{table}\n")
sink()
```

Appendix A8 - SQL Question 2: COVID-19 Crash Cumulative Returns

Author: SISON, Aaron Joshua Estacio Measures crisis resilience during the February to March 2020 COVID-19 market crash by computing cumulative returns for each asset.

```
# Q2: COVID-19 Crash Cumulative Returns -- SISON, Aaron Joshua Estacio

q2_raw <- fin_data |>
  select(Date, Asset, Daily_Return) |>
  collect() |>
  mutate(Date = as.Date(Date))

q2 <- q2_raw |>
  filter(Date >= as.Date("2020-02-01") & Date <= as.Date("2020-03-31")) |>
  filter(!is.na(Daily_Return)) |>
  group_by(Asset) |>
  arrange(Date) |>
  mutate(Cumulative_Return = cumprod(1 + Daily_Return) - 1)

q2_summary <- q2 |>
  summarise(
    Start_Date = min(Date),
    End_Date = max(Date),
```

```
    Total_Cumulative_Return = last(Cumulative_Return),
    Observations = n()
  )

safe_kable(q2_summary, digits=4, caption="Q2 Summary Data Output")
```

Table 8: Q2 Summary Data Output

Asset	Start_Date	End_Date	Total_Cumulative_Return	Observations
AAPL	2020-02-03	2020-03-31	-0.1611	42
BDO	2020-02-03	2020-03-31	-0.0533	42
BTC	2020-02-03	2020-03-31	-0.3114	42
SPY	2020-02-03	2020-03-31	-0.1921	42

```
# Write LaTeX table to file
sink("../reports/sections/q2_table.tex")
cat("\\begin{table}[H]\n\\centering\n")
cat("\\caption{Q2 Summary Data Output}\n")
cat("\\label{tab:q2}\n")
cat("\\begin{tabular}{lllrr}\n")
cat("\\toprule\n")
cat("Asset & Start\\_Date & End\\_Date & Total\\_Cumulative\\_Return &
 ↪  Observations \\\\
")
cat("\\midrule\n")
for (i in 1:nrow(q2_summary)) {
  cat(sprintf("%s & %s & %s & %.4f & %d \\\\
",
    q2_summary$Asset[i], q2_summary$Start_Date[i], q2_summary$End_Date[i],
    q2_summary$Total_Cumulative_Return[i], q2_summary$Observations[i]))
}
cat("\\bottomrule\n\\end{tabular}\n\\end{table}\n")
sink()
```

Appendix A9 - SQL Question 3: Pairwise Correlation of Daily Returns

Author: SISON, Aaron Joshua Estacio Calculates the Pearson correlation coefficients for daily returns across all asset pairs and displays the correlation matrix.

```
# Q3: Pairwise Correlation of Daily Returns -- SISON, Aaron Joshua Estacio

returns_wide <- fin_data |>
```

```
  select(Date, Asset, Daily_Return) |>
  collect() |>
  pivot_wider(names_from = Asset, values_from = Daily_Return)

cor_matrix <- returns_wide |>
  select(AAPL, BDO, BTC, SPY) |>
  cor(use = "pairwise.complete.obs")

cor_long <- as.data.frame(cor_matrix) |>
  tibble::rownames_to_column("Asset1") |>
  tidyr::pivot_longer(-Asset1, names_to = "Asset2", values_to =
 ↪  "Correlation") |>
  mutate(Asset1 = factor(Asset1, levels = rev(c("AAPL", "BDO", "BTC",
   ↪  "SPY"))),
         Asset2 = factor(Asset2, levels = c("AAPL", "BDO", "BTC", "SPY")))

safe_kable(round(cor_matrix, 4), caption="Q3 Correlation Matrix Output")
```

Table 9: Q3 Correlation Matrix Output

	AAPL	BDO	BTC	SPY
AAPL	1.0000	0.1072	0.2899	0.7869
BDO	0.1072	1.0000	0.0880	0.1309
BTC	0.2899	0.0880	1.0000	0.3764
SPY	0.7869	0.1309	0.3764	1.0000

```
# Write LaTeX table to file
sink("../reports/sections/q3_table.tex")
cat("\\begin{table}[H]\n\\centering\n")
cat("\\caption{Q3 Correlation Matrix Output}\n")
cat("\\label{tab:q3}\n")
cat("\\begin{tabular}{lrrrr}\n")
cat("\\toprule\n")
cat("  & AAPL & BDO & BTC & SPY \\\\
")
cat("\\midrule\n")
for (a in rownames(cor_matrix)) {
  cat(sprintf("%s & %.4f & %.4f & %.4f & %.4f \\\\
",
    a, cor_matrix[a,"AAPL"], cor_matrix[a,"BDO"], cor_matrix[a,"BTC"],
 ↪  cor_matrix[a,"SPY"]))
}
cat("\\bottomrule\n\\end{tabular}\n\\end{table}\n")
sink()
```

Appendix A10 - SQL Question 4: Asset Ranking by Total Return

Author: CRUZ, Ricardo Miguel Iñigo Ranks assets by total price return and annualized return over the full 2020-2025 sample period.

```
# Q4: Asset Ranking by Total Return -- CRUZ, Ricardo Miguel Inigo

q4_raw <- fin_data |>
  select(Date, Asset, Close, Daily_Return) |>
  collect() |>
  mutate(Date = as.Date(Date))

q4 <- q4_raw |>
  filter(!is.na(Close), !is.na(Daily_Return)) |>
  group_by(Asset) |>
  arrange(Date) |>
  summarise(
    Start_Date = min(Date),
    End_Date = max(Date),
    Start_Close = first(Close),
    End_Close = last(Close),
    Total_Return = (End_Close / Start_Close) - 1,
    Annualized_Return = (1 + Total_Return)^(365.25 / as.numeric(End_Date -
    ↪  Start_Date)) - 1,
    Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
    SD_Daily_Return = sd(Daily_Return, na.rm = TRUE),
    Observations = n(),
    .groups = "drop"
  ) |>
  arrange(desc(Total_Return))

q4_plot <- q4 |>
  mutate(Asset = factor(Asset, levels = q4$Asset[order(q4$Total_Return)]))

safe_kable(q4, digits=4, caption="Q4 Data Output")
```

Table 10: Q4 Data Output

Asset	Start_Date	End_Date	Start_Close	End_Close	Total_Return	Annualized_Return	Mean_Daily_Return	SD_Daily_Return	Observations
BTC	2020-01-01	2025-12-31	7200.17	87508.83	11.1537	0.5164	0.0023	0.0380	1566
AAPL	2020-01-01	2025-12-31	72.33	271.36	2.7517	0.2466	0.0012	0.0193	1566
SPY	2020-01-01	2025-12-31	296.13	678.32	1.2906	0.1482	0.0007	0.0127	1566
BDO	2020-01-01	2025-12-31	127.35	134.60	0.0569	0.0093	0.0006	0.0204	1566

```
# Write LaTeX table to file
sink("../reports/sections/q4_table.tex")
cat("\\begin{table}[H]\n\\centering\n")
cat("\\caption{Q4 Data Output}\n")
cat("\\label{tab:q4}\n")
cat("\\begin{tabular}{lllrrrrrrr}\n")
cat("\\toprule\n")
cat("Asset & Start\\_Date & End\\_Date & Start\\_Close & End\\_Close &
 ↪  Total\\_Return & Annualized\\_Return & Mean\\_Daily\\_Return &
 ↪  SD\\_Daily\\_Return & Obs \\\\
")
cat("\\midrule\n")
for (i in 1:nrow(q4)) {
  cat(sprintf("%s & %s & %s & %.2f & %.2f & %.4f & %.4f & %.4f & %.4f & %d
   ↪  \\\\
",
    q4$Asset[i], q4$Start_Date[i], q4$End_Date[i], q4$Start_Close[i],
 ↪  q4$End_Close[i],
    q4$Total_Return[i], q4$Annualized_Return[i], q4$Mean_Daily_Return[i],
 ↪  q4$SD_Daily_Return[i],
    q4$Observations[i]))
}
cat("\\bottomrule\n\\end{tabular}\n\\end{table}\n")
sink()
```

Appendix A11 - SQL Question 5: Best and Worst Trading Days

Author: CRUZ, Ricardo Miguel Iñigo Identifies extreme single-day gains and losses for each asset to assess tail risk.

```
# Q5: Best and Worst Trading Days -- CRUZ, Ricardo Miguel Inigo

q5_raw <- fin_data |>
  select(Date, Asset, Daily_Return) |>
  collect() |>
  mutate(Date = as.Date(Date))

q5 <- q5_raw |>
  filter(!is.na(Daily_Return)) |>
  group_by(Asset) |>
  summarise(
    Best_Date = Date[which.max(Daily_Return)],
```

```
    Best_Daily_Return = max(Daily_Return, na.rm = TRUE),
    Worst_Date = Date[which.min(Daily_Return)],
    Worst_Daily_Return = min(Daily_Return, na.rm = TRUE),
    Return_Range = Best_Daily_Return - Worst_Daily_Return,
    Observations = n(),
    .groups = "drop"
  ) |>
  arrange(Worst_Daily_Return)

safe_kable(q5, digits=4, caption="Q5 Data Output")
```

Table 11: Q5 Data Output

Asset	Best_Date	Best_Daily_Return	Worst_Date	Worst_Daily_Return	Return_Range	Observations
BTC	2021-02-08	0.2111	2020-03-12	-0.3717	0.5828	1566
BDO	2020-03-26	0.1570	2020-03-16	-0.1339	0.2909	1566
AAPL	2025-04-09	0.1533	2020-03-16	-0.1286	0.2819	1566
SPY	2025-04-09	0.1050	2020-03-16	-0.1094	0.2144	1566

```
# Write LaTeX table to file
sink("../reports/sections/q5_table.tex")
cat("\\begin{table}[H]\n\\centering\n")
cat("\\caption{Q5 Data Output}\n")
cat("\\label{tab:q5}\n")
cat("\\begin{tabular}{lllrrrr}\n")
cat("\\toprule\n")
cat("Asset & Best\\_Date & Best\\_Daily\\_Return & Worst\\_Date &
 ↪  Worst\\_Daily\\_Return & Return\\_Range & Obs \\\\
")
cat("\\midrule\n")
for (i in 1:nrow(q5)) {
  cat(sprintf("%s & %s & %.4f & %s & %.4f & %.4f & %d \\\\
",
    q5$Asset[i], q5$Best_Date[i], q5$Best_Daily_Return[i],
    q5$Worst_Date[i], q5$Worst_Daily_Return[i], q5$Return_Range[i],
 ↪  q5$Observations[i]))
}
cat("\\bottomrule\n\\end{tabular}\n\\end{table}\n")
sink()
```

Appendix A12 - SQL Question 6: Moving Average Crossover Analysis

Author: CRUZ, Ricardo Miguel Iñigo Implements a 20-day and 60-day simple moving average crossover strategy to analyze momentum regime shifts.

```
# Q6: Moving Average Crossover Analysis -- CRUZ, Ricardo Miguel Inigo

q6_raw <- fin_data |>
  select(Date, Asset, Close, Daily_Return) |>
  collect() |>
  mutate(Date = as.Date(Date))

q6_ma <- q6_raw |>
  filter(!is.na(Close)) |>
  group_by(Asset) |>
  arrange(Date) |>
  mutate(
    MA_20 = as.numeric(stats::filter(Close, rep(1 / 20, 20), sides = 1)),
    MA_60 = as.numeric(stats::filter(Close, rep(1 / 60, 60), sides = 1)),
    Signal = case_when(
      MA_20 > MA_60 ~ "Bullish",
      MA_20 < MA_60 ~ "Bearish",
      TRUE ~ "Neutral"
    )
  ) |>
  ungroup()

q6 <- q6_ma |>
  filter(!is.na(MA_20), !is.na(MA_60), !is.na(Daily_Return)) |>
  group_by(Asset, Signal) |>
  summarise(
    Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
    SD_Daily_Return = sd(Daily_Return, na.rm = TRUE),
    Observations = n(),
    .groups = "drop"
  ) |>
  arrange(Asset, desc(Mean_Daily_Return))

q6_latest_signal <- q6_ma |>
  filter(!is.na(MA_20), !is.na(MA_60)) |>
  group_by(Asset) |>
  slice_max(Date, n = 1, with_ties = FALSE) |>
  ungroup() |>
  select(Asset, Date, Close, MA_20, MA_60, Signal) |>
```

```
  arrange(Asset)

safe_kable(q6, digits=4, caption="Q6 Data Output")
```

Table 12: Q6 Data Output

Asset	Signal	Mean_Daily_Return	SD_Daily_Return	Observations
AAPL	Bullish	0.0015	0.0159	922
AAPL	Bearish	0.0013	0.0209	585
BDO	Bearish	0.0009	0.0206	795
BDO	Bullish	0.0006	0.0180	712
BTC	Bullish	0.0035	0.0357	867
BTC	Bearish	0.0009	0.0375	640
SPY	Bearish	0.0018	0.0165	391
SPY	Bullish	0.0006	0.0087	1116

```
# Write LaTeX table to file
sink("../reports/sections/q6_table.tex")
cat("\\begin{table}[H]\n\\centering\n")
cat("\\caption{Q6 Data Output}\n")
cat("\\label{tab:q6}\n")
cat("\\begin{tabular}{llrrr}\n")
cat("\\toprule\n")
cat("Asset & Signal & Mean\\_Daily\\_Return & SD\\_Daily\\_Return & Obs \\\\
")
cat("\\midrule\n")
for (i in 1:nrow(q6)) {
  cat(sprintf("%s & %s & %.4f & %.4f & %d \\\\
",
    q6$Asset[i], q6$Signal[i], q6$Mean_Daily_Return[i],
 ↪  q6$SD_Daily_Return[i], q6$Observations[i]))
}
cat("\\bottomrule\n\\end{tabular}\n\\end{table}\n")
sink()
```

Appendix A13 - SQL Question 7: Volume and Volatility Relationship

Author: CRUZ, Ricardo Miguel Iñigo Tests whether elevated trading volume systematically coincides with higher volatility by partitioning days into high-volume and normal-volume regimes.

```
# Q7: Volume and Volatility Relationship -- CRUZ, Ricardo Miguel Inigo
```

```r
volume_lookup <- assets_all |>
  select(Date, Asset, Volume) |>
  mutate(
    Date = as.Date(Date),
    Volume = as.numeric(Volume)
  )

q7_raw <- fin_data |>
  select(Date, Asset, Daily_Return) |>
  collect() |>
  mutate(
    Date = as.Date(Date),
    Daily_Return = as.numeric(Daily_Return)
  ) |>
  left_join(volume_lookup, by = c("Date", "Asset")) |>
  filter(
    !is.na(Daily_Return),
    !is.na(Volume),
    Volume > 0
  )

q7 <- q7_raw |>
  group_by(Asset) |>
  mutate(
    Volume_Threshold = quantile(Volume, 0.75, na.rm = TRUE),
    Volume_Regime = if_else(
      Volume >= Volume_Threshold,
      "High Volume Days",
      "Normal Volume Days"
    ),
    Absolute_Return = abs(Daily_Return)
  ) |>
  ungroup() |>
  group_by(Asset, Volume_Regime) |>
  summarise(
    Observations = n(),
    Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
    Mean_Absolute_Return = mean(Absolute_Return, na.rm = TRUE),
    Volatility = sd(Daily_Return, na.rm = TRUE),
    Average_Volume = mean(Volume, na.rm = TRUE),
    .groups = "drop"
  ) |>
  arrange(Asset, Volume_Regime)

safe_kable(q7, digits=4, caption="Q7 Data Output")
```

Table 13: Q7 Data Output

Asset	Volume_Regime	Observations	Mean_Daily_Return	Mean_Absolute_Return	Volatility	Average_Volume
AAPL	High Volume Days	377	0.0018	0.0233	0.0319	152425015
AAPL	Normal Volume Days	1131	0.0011	0.0099	0.0132	61908113
BDO	High Volume Days	367	0.0014	0.0204	0.0286	6581172
BDO	Normal Volume Days	1101	0.0004	0.0130	0.0178	2550776
BTC	High Volume Days	392	0.0014	0.0358	0.0533	69794321551
BTC	Normal Volume Days	1174	0.0027	0.0225	0.0314	29942820671
SPY	High Volume Days	377	-0.0027	0.0156	0.0219	129389767
SPY	Normal Volume Days	1131	0.0018	0.0058	0.0076	63707966

```
# Write LaTeX table to file
sink("../reports/sections/q7_table.tex")
cat("\\begin{table}[H]\n\\centering\n")
cat("\\caption{Q7 Data Output}\n")
cat("\\label{tab:q7}\n")
cat("\\begin{tabular}{llrrrrr}\n")
cat("\\toprule\n")
cat("Asset & Volume\\_Regime & Obs & Mean\\_Daily\\_Return &
 ↪  Mean\\_Abs\\_Return & Volatility & Avg\\_Volume \\\\
")
cat("\\midrule\n")
for (i in 1:nrow(q7)) {
  cat(sprintf("%s & %s & %d & %.4f & %.4f & %.4f & %.0f \\\\
",
    q7$Asset[i], q7$Volume_Regime[i], q7$Observations[i],
    q7$Mean_Daily_Return[i], q7$Mean_Absolute_Return[i],
    q7$Volatility[i], q7$Average_Volume[i]))
}
cat("\\bottomrule\n\\end{tabular}\n\\end{table}\n")
sink()
```

Appendix A14 - SQL Question 8: VIX Regime Analysis

Author: SISON, Aaron Joshua Estacio Assesses asset performance during periods of high versus low market fear by segregating trading days based on VIX levels.

```
# Q8: VIX Regime Analysis -- SISON, Aaron Joshua Estacio

q8_raw <- fin_data |>
  filter(!is.na(VIX)) |>
  select(Asset, Date, Daily_Return, VIX) |>
  collect() |>
```

```r
  mutate(Date = as.Date(Date))

q8 <- q8_raw |>
  mutate(
    VIX_Regime = case_when(
      VIX > 30 ~ "High Volatility (VIX > 30)",
      VIX < 20 ~ "Low Volatility (VIX < 20)",
      TRUE ~ "Moderate (20 <= VIX <= 30)"
    )
  ) |>
  group_by(Asset, VIX_Regime) |>
  summarise(
    Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
    SD_Daily_Return = sd(Daily_Return, na.rm = TRUE),
    Observations = n(),
    .groups = "drop"
  ) |>
  arrange(Asset, VIX_Regime)

q8_plot <- q8 |>
  filter(VIX_Regime %in% c("High Volatility (VIX > 30)", "Low Volatility
   ↪  (VIX < 20)")) |>
  mutate(VIX_Regime = recode(VIX_Regime,
    "High Volatility (VIX > 30)" = "VIX > 30 (High Fear)",
    "Low Volatility (VIX < 20)" = "VIX < 20 (Low Fear)"))

safe_kable(q8, digits=4, caption="Q8 Data Output")
```

Table 14: Q8 Data Output

Asset	VIX_Regime	Mean_Daily_Return	SD_Daily_Return	Observations
AAPL	High Volatility (VIX > 30)	-0.0051	0.0380	150
AAPL	Low Volatility (VIX < 20)	0.0025	0.0127	877
AAPL	Moderate (20 <= VIX <= 30)	0.0009	0.0200	539
BDO	High Volatility (VIX > 30)	-0.0015	0.0348	150
BDO	Low Volatility (VIX < 20)	0.0008	0.0171	877
BDO	Moderate (20 <= VIX <= 30)	0.0010	0.0199	539
BTC	High Volatility (VIX > 30)	-0.0031	0.0575	150
BTC	Low Volatility (VIX < 20)	0.0029	0.0312	877
BTC	Moderate (20 <= VIX <= 30)	0.0029	0.0412	539
SPY	High Volatility (VIX > 30)	-0.0041	0.0298	150
SPY	Low Volatility (VIX < 20)	0.0017	0.0066	877
SPY	Moderate (20 <= VIX <= 30)	0.0004	0.0119	539

```r
# Write LaTeX table to file
sink("../reports/sections/q8_table.tex")
```

```
cat("\\begin{table}[H]\n\\centering\n")
cat("\\caption{Q8 Data Output}\n")
cat("\\label{tab:q8}\n")
cat("\\begin{tabular}{llrrr}\n")
cat("\\toprule\n")
cat("Asset & VIX\\_Regime & Mean\\_Daily\\_Return & SD\\_Daily\\_Return &
 ↪  Obs \\\\
")
cat("\\midrule\n")
for (i in 1:nrow(q8)) {
  cat(sprintf("%s & %s & %.4f & %.4f & %d \\\\
",
    q8$Asset[i], q8$VIX_Regime[i], q8$Mean_Daily_Return[i],
    q8$SD_Daily_Return[i], q8$Observations[i]))
}
cat("\\bottomrule\n\\end{tabular}\n\\end{table}\n")
sink()
```

Appendix A15 - SQL Question 9: Interest Rate Sensitivity

Author: SISON, Aaron Joshua Estacio Measures asset sensitivity to changes in the 10-year Treasury yield by comparing mean returns on days when yields rise versus fall.

```
# Q9: Interest Rate Sensitivity -- SISON, Aaron Joshua Estacio

q9_raw <- fin_data |>
  filter(!is.na(DGS10)) |>
  select(Asset, Date, Daily_Return, DGS10) |>
  collect() |>
  mutate(Date = as.Date(Date))

q9 <- q9_raw |>
  group_by(Asset) |>
  arrange(Date) |>
  mutate(DGS10_Change = DGS10 - lag(DGS10)) |>
  ungroup() |>
  filter(!is.na(DGS10_Change)) |>
  mutate(
    Yield_Direction = case_when(
      DGS10_Change > 0 ~ "Yields Rising",
      DGS10_Change < 0 ~ "Yields Falling",
      TRUE ~ "No Change"
```

```
    )
  ) |>
  group_by(Asset, Yield_Direction) |>
  summarise(
    Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
    SD_Daily_Return = sd(Daily_Return, na.rm = TRUE),
    Observations = n(),
    .groups = "drop"
  ) |>
  arrange(Asset, Yield_Direction)

q9_plot <- q9 |>
  filter(Yield_Direction %in% c("Yields Rising", "Yields Falling"))

safe_kable(q9, digits=4, caption="Q9 Data Output")
```

Table 15: Q9 Data Output

Asset	Yield_Direction	Mean_Daily_Return	SD_Daily_Return	Observations
AAPL	No Change	0.0019	0.0159	169
AAPL	Yields Falling	0.0007	0.0195	684
AAPL	Yields Rising	0.0015	0.0197	712
BDO	No Change	0.0015	0.0163	169
BDO	Yields Falling	0.0003	0.0227	684
BDO	Yields Rising	0.0008	0.0189	712
BTC	No Change	0.0053	0.0407	169
BTC	Yields Falling	0.0016	0.0360	684
BTC	Yields Rising	0.0023	0.0394	712
SPY	No Change	0.0003	0.0081	169
SPY	Yields Falling	-0.0001	0.0129	684
SPY	Yields Rising	0.0015	0.0133	712

```
# Write LaTeX table to file
sink("../reports/sections/q9_table.tex")
cat("\\begin{table}[H]\n\\centering\n")
cat("\\caption{Q9 Data Output}\n")
cat("\\label{tab:q9}\n")
cat("\\begin{tabular}{llrrr}\n")
cat("\\toprule\n")
cat("Asset & Yield\\_Direction & Mean\\_Daily\\_Return & SD\\_Daily\\_Return
 ↪  & Obs \\\\
")
cat("\\midrule\n")
for (i in 1:nrow(q9)) {
  cat(sprintf("%s & %s & %.4f & %.4f & %d \\\\
",
```

```
    q9$Asset[i], q9$Yield_Direction[i], q9$Mean_Daily_Return[i],
    q9$SD_Daily_Return[i], q9$Observations[i]))
}
cat("\\bottomrule\n\\end{tabular}\n\\end{table}\n")
sink()
```

Appendix A16 - SQL Question 10: Inflation Hedge Performance

Author: SISON, Aaron Joshua Estacio Evaluates which assets served as effective inflation hedges during the June 2021 to February 2023 high-inflation period (CPI YoY above 5%).

```
# Q10: Inflation Hedge Performance -- SISON, Aaron Joshua Estacio

q10_raw <- fin_data |>
  select(Date, Asset, Daily_Return, CPI_YoY) |>
  collect() |>
  mutate(Date = as.Date(Date))

q10 <- q10_raw |>
  filter(!is.na(CPI_YoY) & CPI_YoY > 0.05) |>
  filter(!is.na(Daily_Return)) |>
  group_by(Asset) |>
  arrange(Date) |>
  mutate(Cumulative_Return = cumprod(1 + Daily_Return) - 1) |>
  summarise(
    Start_Date = min(Date),
    End_Date = max(Date),
    Start_CPI_YoY = first(CPI_YoY),
    End_CPI_YoY = last(CPI_YoY),
    Total_Cumulative_Return = last(Cumulative_Return),
    Mean_Daily_Return = mean(Daily_Return, na.rm = TRUE),
    Observations = n()
  ) |>
  arrange(desc(Total_Cumulative_Return))

q10_plot <- q10 |>
  mutate(Asset = factor(Asset, levels =
   ↪  q10$Asset[order(q10$Total_Cumulative_Return)]))

safe_kable(q10, digits=4, caption="Q10 Data Output")
```

Table 16: Q10 Data Output

Asset	Start_Date	End_Date	Start_CPI_YoY	End_CPI_YoY	Total_Cumulative_Return	Mean_Daily_Return	Observations
BDO	2021-06-01	2023-02-28	0.053	0.0596	0.4531	0.0010	456
AAPL	2021-06-01	2023-02-28	0.053	0.0596	0.2865	0.0007	456
SPY	2021-06-01	2023-02-28	0.053	0.0596	0.0243	0.0001	456
BTC	2021-06-01	2023-02-28	0.053	0.0596	-0.3800	-0.0002	456

```
# Write LaTeX table to file
sink("../reports/sections/q10_table.tex")
cat("\\begin{table}[H]\n\\centering\n")
cat("\\caption{Q10 Data Output}\n")
cat("\\label{tab:q10}\n")
cat("\\begin{tabular}{lllrrrrr}\n")
cat("\\toprule\n")
cat("Asset & Start\\_Date & End\\_Date & Start\\_CPI\\_YoY & End\\_CPI\\_YoY
 ↪  & Total\\_Cumulative\\_Return & Mean\\_Daily\\_Return & Obs \\\\
")
cat("\\midrule\n")
for (i in 1:nrow(q10)) {
  cat(sprintf("%s & %s & %s & %.4f & %.4f & %.4f & %.4f & %d \\\\
",
    q10$Asset[i], q10$Start_Date[i], q10$End_Date[i],
    q10$Start_CPI_YoY[i], q10$End_CPI_YoY[i],
    q10$Total_Cumulative_Return[i], q10$Mean_Daily_Return[i],
    q10$Observations[i]))
}
cat("\\bottomrule\n\\end{tabular}\n\\end{table}\n")
sink()
```

Appendix A17 - Descriptive Statistics

Author: GALEDO, Enrique Lorenzo Hermoso Computes the full 9-statistic descriptive table across all final-dataset variables, the correlation matrix of daily returns and macroeconomic series, faceted density plots, and individual time series plots for each variable.

```
# Descriptive Statistics - GALEDO, Enrique Lorenzo Hermoso

# Select all numeric columns from final_dataset (wide format), drop Date
numeric_cols <- final_dataset |>
  select(-Date) |>
  select(where(is.numeric))

calc_skewness <- function(x) {
  x <- x[!is.na(x)]
```

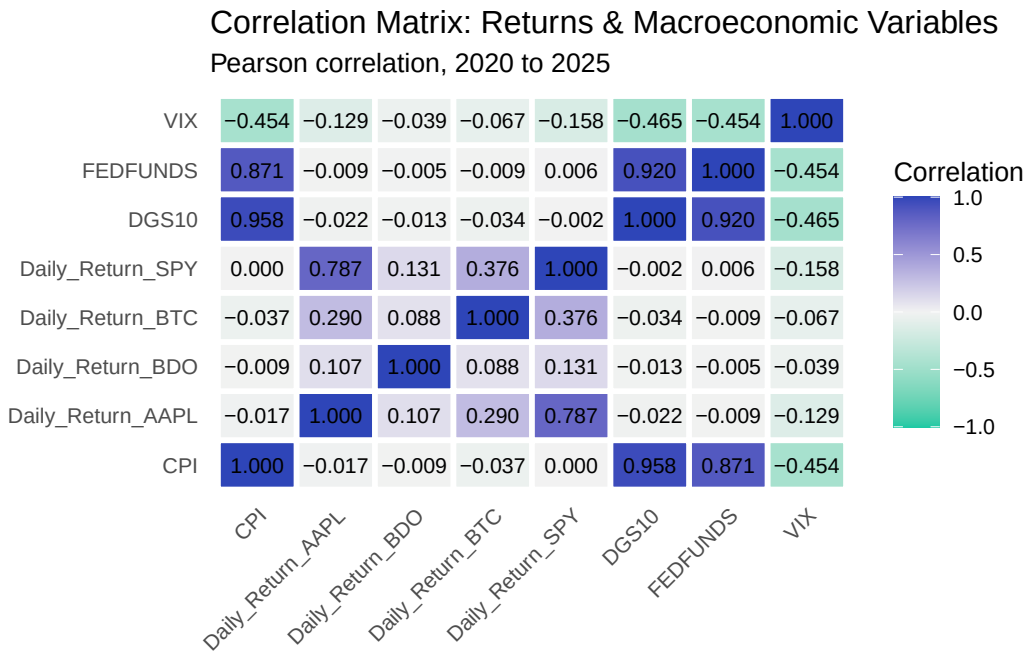
```r
  n <- length(x)
  if (n < 3) return(NA_real_)
  m <- mean(x)
  s <- sd(x)
  if (s == 0) return(NA_real_)
  (sum((x - m)^3) / n) / (s^3)
}

calc_kurtosis <- function(x) {
  x <- x[!is.na(x)]
  n <- length(x)
  if (n < 4) return(NA_real_)
  m <- mean(x)
  s <- sd(x)
  if (s == 0) return(NA_real_)
  (sum((x - m)^4) / n) / (s^2)^2 - 3
}

stat_names <- c("Mean", "Median", "Variance", "Std_Dev", "Min", "Max",
 ↪  "Range", "Skewness", "Kurtosis")
desc_stats <- data.frame(Statistic = stat_names, row.names = NULL,
 ↪  stringsAsFactors = FALSE)

for (col_name in names(numeric_cols)) {
  x <- numeric_cols[[col_name]]
  x_clean <- x[!is.na(x)]
  desc_stats[[col_name]] <- c(
    mean(x_clean),
    median(x_clean),
    var(x_clean),
    sd(x_clean),
    min(x_clean),
    max(x_clean),
    max(x_clean) - min(x_clean),
    calc_skewness(x_clean),
    calc_kurtosis(x_clean)
  )
}

write.csv(desc_stats, file.path(data_dir, "descriptive_stats.csv"),
 ↪  row.names = FALSE, na = "")

safe_kable(desc_stats, digits = 4, caption = paste("Descriptive Statistics -
 ↪  Final Dataset Variables (", ncol(numeric_cols), " columns)", sep = ""))
```

Table 17: Descriptive Statistics — Final Dataset Variables (12 columns)

Statistic	Close_AAPL	Close_BDO	Close_BTC	Close_SPY	Daily_Return_AAPL	Daily_Return_BDO	Daily_Return_BTC	Daily_Return_SPY	CPI	DGS10	FEDFUNDS	VIX
Mean	164.7239	118.6006	47212.9191	438.7232	0.0012	0.0006	0.0023	0.0007	293.6122	2.9587	2.7553	20.9324
Median	163.0800	123.2500	39934.6650	414.3350	0.0000	0.0000	0.0007	0.0004	298.8320	3.5700	3.7800	19.0000
Variance	2469.6072	696.9352	988235218.6940	11746.4060	0.0004	0.0004	0.0014	0.0002	523.2442	1.9771	4.9038	61.6242
Std_Dev	49.6951	26.3995	31436.2087	108.3808	0.0193	0.0204	0.0380	0.0127	22.8745	1.4061	2.2145	7.8501
Min	54.1600	70.2900	4970.7900	204.4200	-0.1286	-0.1339	-0.3717	-0.1094	255.8020	0.5200	0.0500	11.8600
Max	285.6600	168.0000	124752.5300	686.7300	0.1533	0.1570	0.2111	0.1050	326.0310	4.9800	5.3300	82.6900
Range	231.5000	97.7100	119781.7400	482.3100	0.2819	0.2909	0.5828	0.2144	70.2290	4.4600	5.2800	70.8300
Skewness	0.0927	-0.0825	0.7299	0.4766	0.3335	0.4081	-0.4464	-0.2729	-0.3336	-0.4161	-0.1667	2.6170
Kurtosis	-0.4105	-1.2613	-0.5189	-0.5565	7.8268	6.6348	9.2965	14.3788	-1.3422	-1.4613	-1.7565	11.9357

Correlation Matrix: Returns & Macroeconomic Variables

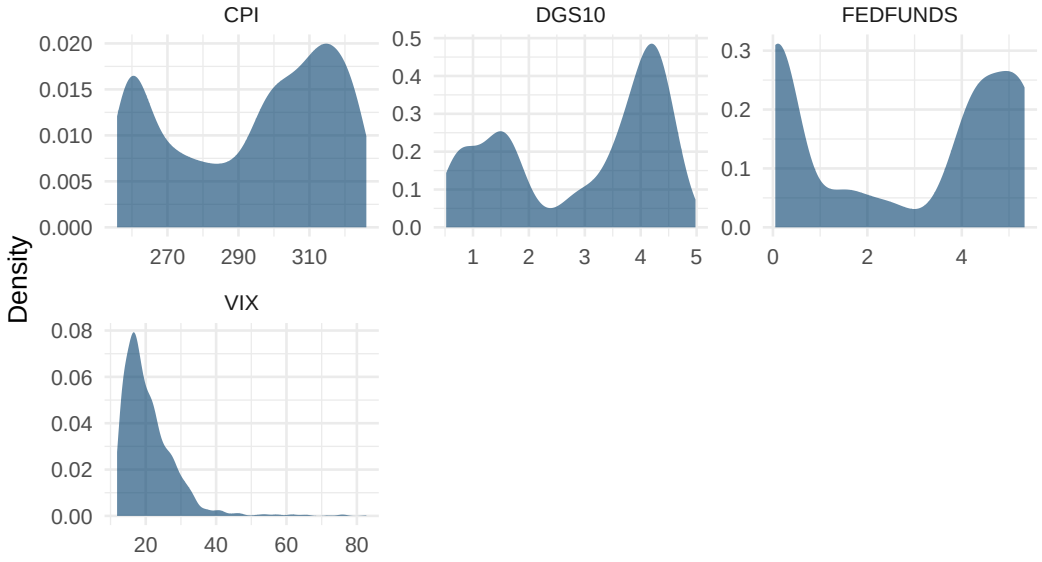
```
desc_cor_cols <- final_dataset |>
  select(Daily_Return_AAPL, Daily_Return_BDO, Daily_Return_BTC,
    ↪  Daily_Return_SPY,
         VIX, FEDFUNDS, CPI, DGS10)

desc_cor_matrix <- cor(desc_cor_cols, use = "pairwise.complete.obs")

write.csv(as.data.frame(desc_cor_matrix), file.path(data_dir,
 ↪  "correlation_matrix.csv"), na = "")

cor_long_desc <- as.data.frame(desc_cor_matrix) |>
  tibble::rownames_to_column("Variable1") |>
  pivot_longer(-Variable1, names_to = "Variable2", values_to =
    ↪  "Correlation")

ggplot(cor_long_desc, aes(x = Variable2, y = Variable1, fill = Correlation))
 ↪  +
  geom_tile(color = "white", linewidth = 1) +
  geom_text(aes(label = sprintf("%.3f", Correlation)), size = 2.8) +
  scale_fill_gradient2(low = "#1DC9A4", mid = "#F2F2F2", high = "#2E45B8",
                       midpoint = 0, limits = c(-1, 1)) +
  labs(title = "Correlation Matrix: Returns & Macroeconomic Variables",
       subtitle = "Pearson correlation, 2020 to 2025", x = NULL, y = NULL) +
  theme_minimal(base_size = 10) +
  theme(panel.grid = element_blank(),
        axis.text.x = element_text(angle = 45, hjust = 1))
```



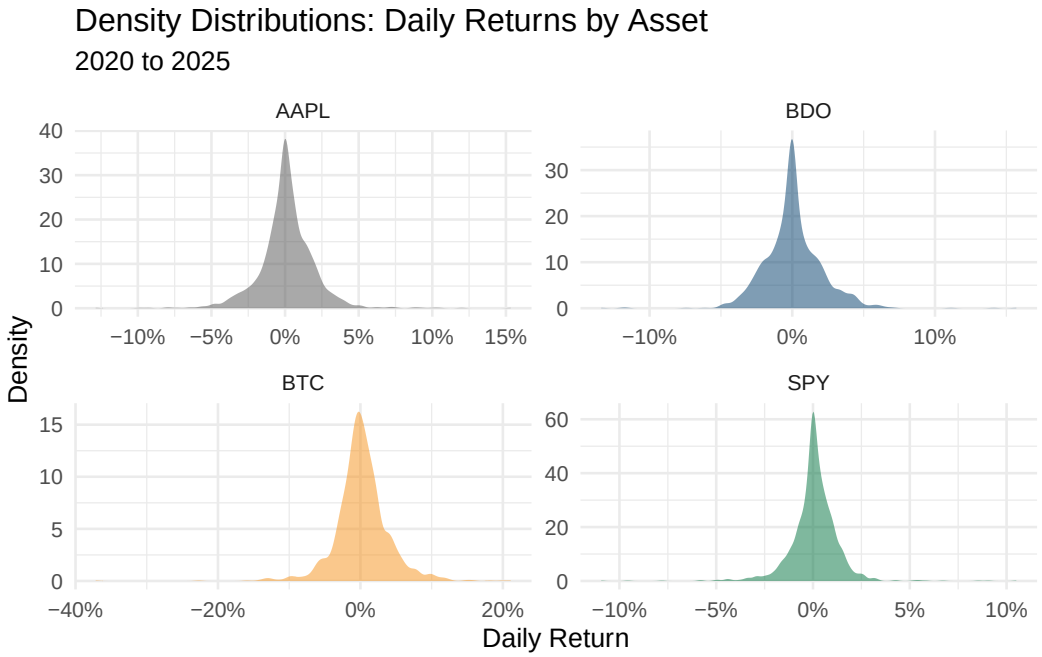

```
ggsave("../reports/figures/fig_desc_correlation.pdf", width = 8, height = 6,
 ↪  device = "pdf")
```

Density Plots

```
# Macroeconomic variables
macro_long <- final_dataset |>
  select(Date, CPI, DGS10, FEDFUNDS, VIX) |>
  pivot_longer(-Date, names_to = "Variable", values_to = "Value") |>
  filter(!is.na(Value))

ggplot(macro_long, aes(x = Value)) +
  geom_density(fill = "#003D6B", alpha = 0.6, color = NA) +
  facet_wrap(~ Variable, scales = "free", ncol = 3) +
  labs(title = "Density Distributions: Macroeconomic Variables",
       subtitle = "2020 to 2025", x = NULL, y = "Density") +
  theme_minimal(base_size = 10)
```



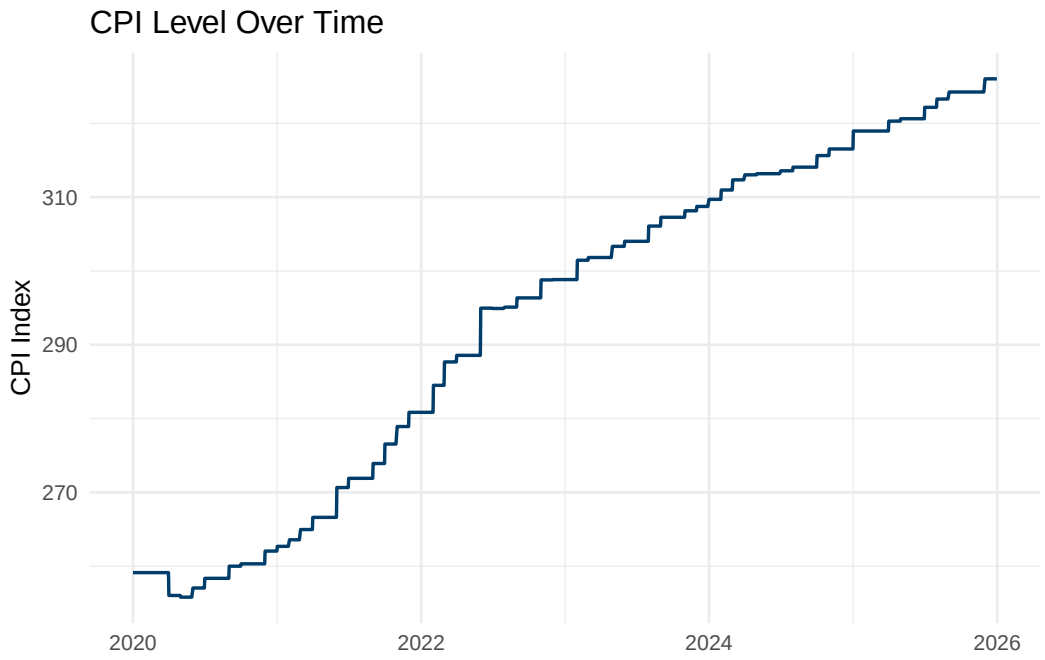

```
ggsave("../reports/figures/fig_desc_density_macro.pdf", width = 9, height =
 ↪  6, device = "pdf")

# Asset daily returns
asset_returns <- sql_long |>
  filter(!is.na(Daily_Return)) |>
  select(Asset, Daily_Return)

ggplot(asset_returns, aes(x = Daily_Return, fill = Asset)) +
  geom_density(alpha = 0.5, color = NA) +
  scale_fill_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
                               BTC = "#F7931A", SPY = "#00733E")) +
  facet_wrap(~ Asset, scales = "free", ncol = 2) +
  scale_x_continuous(labels = scales::percent_format()) +
  labs(title = "Density Distributions: Daily Returns by Asset",
       subtitle = "2020 to 2025", x = "Daily Return", y = "Density") +
  theme_minimal(base_size = 10) +
  theme(legend.position = "none")
```




```
ggsave("../reports/figures/fig_desc_density_returns.pdf", width = 8, height
 ↪  = 6, device = "pdf")
```

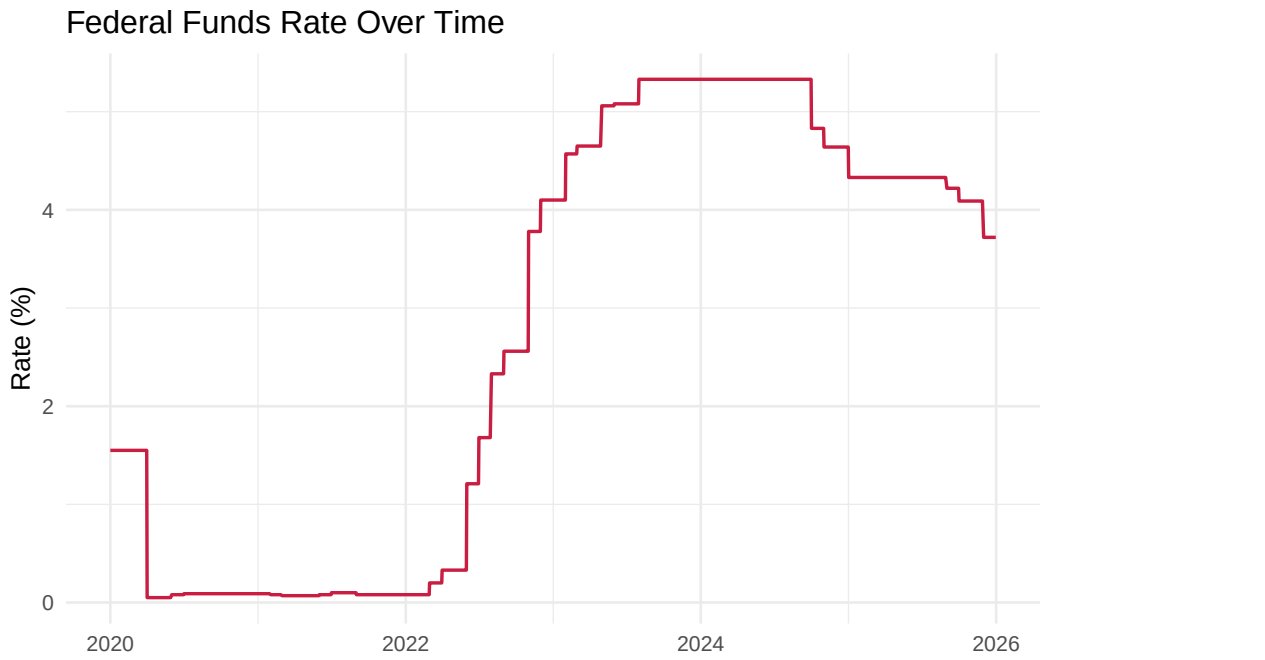
Time Series Plots

```
palette <- c(AAPL = "#555555", BDO = "#003D6B", BTC = "#F7931A", SPY =
 ↪  "#00733E")

# Macroeconomic variables - individual plots

ggplot(cpi, aes(x = Date, y = CPI)) +
  geom_line(color = "#003D6B", linewidth = 0.6) +
  labs(title = "CPI Level Over Time", x = NULL, y = "CPI Index") +
  theme_minimal(base_size = 10)
```

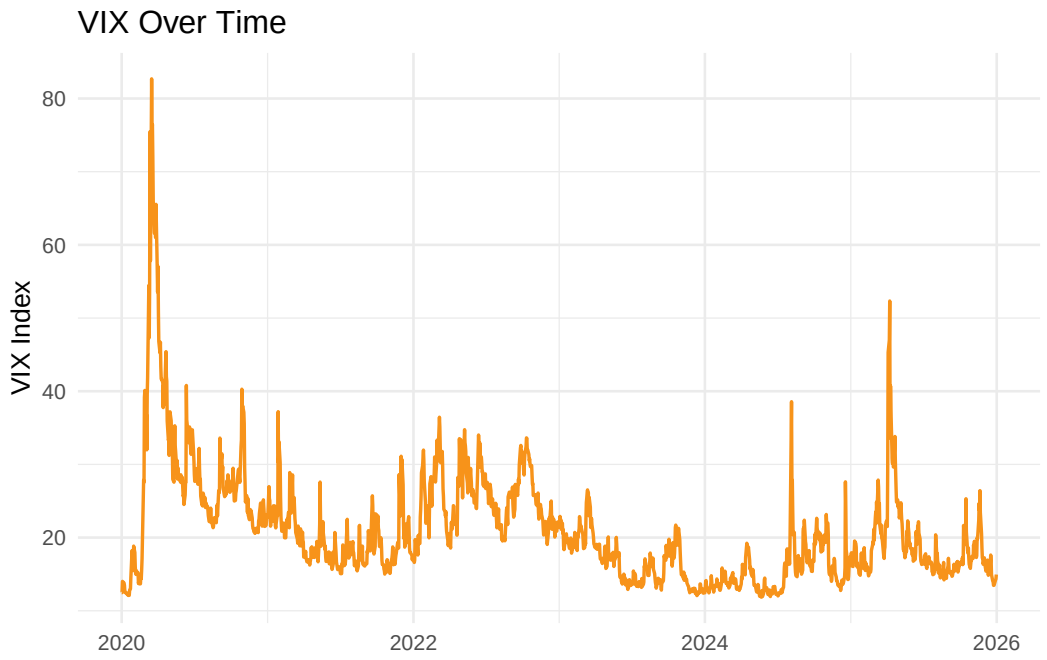



```
ggsave("../reports/figures/fig_desc_ts_cpi.pdf", width = 7, height = 3.5,
 ↪  device = "pdf")

ggplot(dgs10, aes(x = Date, y = DGS10)) +
  geom_line(color = "#00733E", linewidth = 0.6) +
  labs(title = "10-Year Treasury Yield Over Time", x = NULL, y = "Yield
   ↪  (%)") +
  theme_minimal(base_size = 10)
```

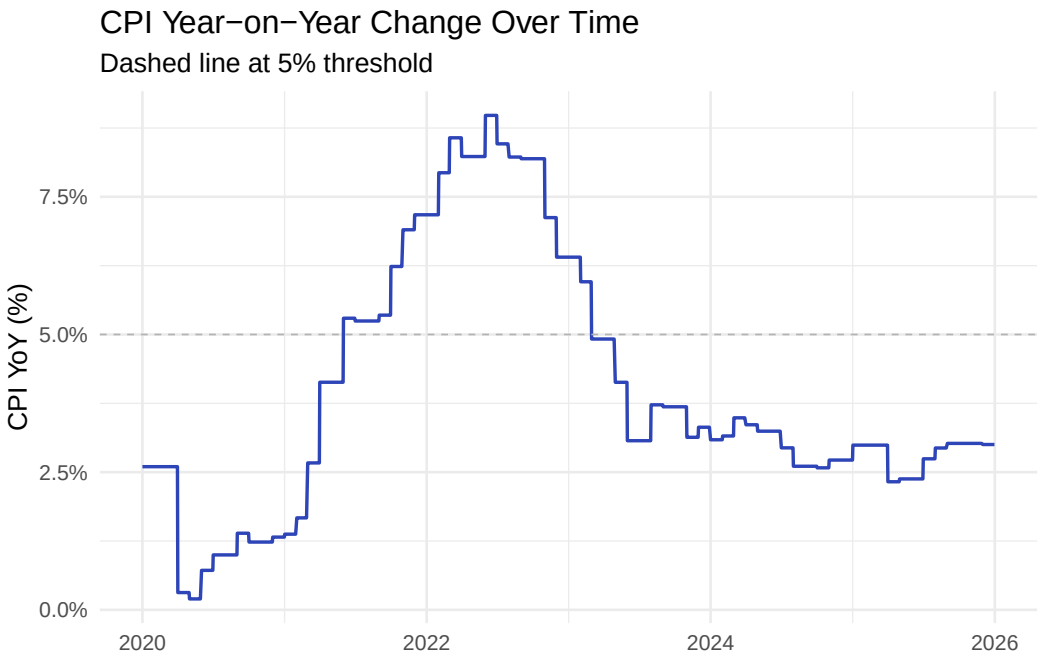
```
ggsave("../reports/figures/fig_desc_ts_dgs10.pdf", width = 7, height = 3.5,
 ↪  device = "pdf")

ggplot(fedfunds, aes(x = Date, y = FEDFUNDS)) +
  geom_line(color = "#C91D42", linewidth = 0.6) +
  labs(title = "Federal Funds Rate Over Time", x = NULL, y = "Rate (%)") +
  theme_minimal(base_size = 10)
```




```
ggsave("../reports/figures/fig_desc_ts_fedfunds.pdf", width = 7, height =
 ↪  3.5, device = "pdf")

ggplot(vix, aes(x = Date, y = VIX)) +
  geom_line(color = "#F7931A", linewidth = 0.6) +
  labs(title = "VIX Over Time", x = NULL, y = "VIX Index") +
  theme_minimal(base_size = 10)
```




```
ggsave("../reports/figures/fig_desc_ts_vix.pdf", width = 7, height = 3.5,
 ↪  device = "pdf")

# CPI_YoY (computed in A6, joined into sql_long - use that source)
cpi_yoy_ts <- sql_long |>
  filter(Asset == "AAPL") |>
  select(Date, CPI_YoY) |>
  filter(!is.na(CPI_YoY)) |>
  distinct(Date, .keep_all = TRUE)

ggplot(cpi_yoy_ts, aes(x = Date, y = CPI_YoY)) +
  geom_line(color = "#2E45B8", linewidth = 0.6) +
  geom_hline(yintercept = 0.05, linetype = "dashed", color = "#B3B3B3",
   ↪  linewidth = 0.3) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(title = "CPI Year-on-Year Change Over Time",
       subtitle = "Dashed line at 5% threshold", x = NULL, y = "CPI YoY
        ↪  (%)") +
  theme_minimal(base_size = 10)
```




```
ggsave("../reports/figures/fig_desc_ts_cpiyoy.pdf", width = 7, height = 3.5,
 ↪  device = "pdf")

# Daily returns per asset - individual plots
for (asset_name in c("AAPL", "BDO", "BTC", "SPY")) {
  asset_data <- sql_long |>
    filter(Asset == asset_name, !is.na(Daily_Return))
  p <- ggplot(asset_data, aes(x = Date, y = Daily_Return)) +
    geom_line(color = palette[asset_name], linewidth = 0.3) +
    scale_y_continuous(labels = scales::percent_format()) +
    labs(title = paste("Daily Returns:", asset_name), x = NULL, y = "Daily
     ↪  Return") +
    theme_minimal(base_size = 10)
  ggsave(sprintf("../reports/figures/fig_desc_ts_returns_%s.pdf",
   ↪  tolower(asset_name)),
         plot = p, width = 7, height = 3.5, device = "pdf")
}
```

TeX Export

```
# Generate LaTeX table snippets for the paper
# Path resolution mirrors data_dir pattern: QMD may render from project root
 ↪  or scripts/
if (dir.exists("scripts")) {
  source("scripts/desc_stats_to_tex.R")
} else {
```

```
  source("desc_stats_to_tex.R")
}
```

Appendix A18 - Data Visualization Charts

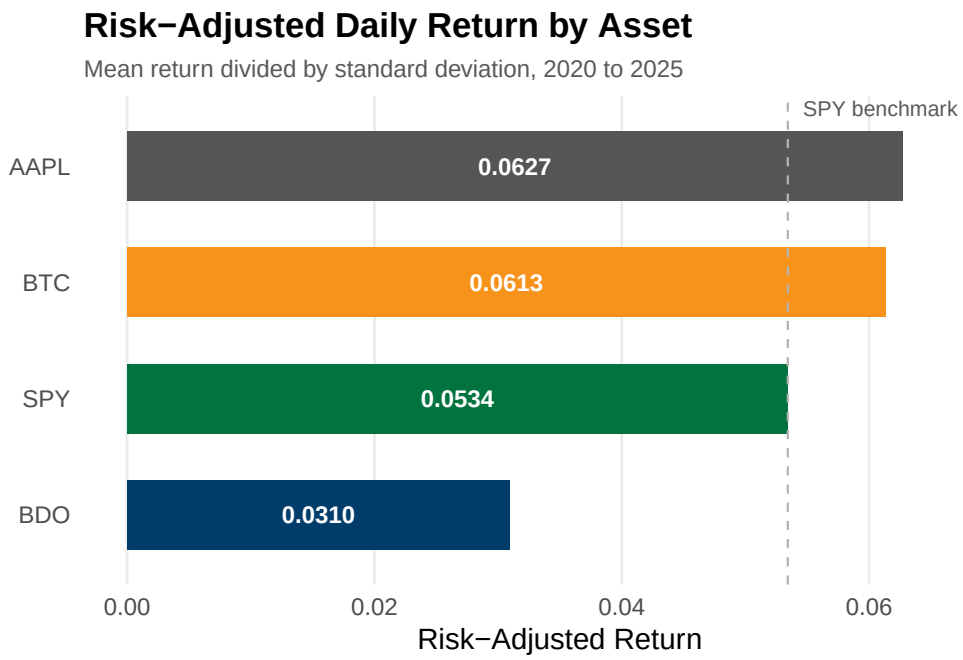
Author: SEBALLOS, Josiah Dweyn Panganiban Contains the ggplot code for all 10 figures corresponding to SQL Questions Q1 through Q10. Each figure is saved as a PDF to the reports/figures/ directory for inclusion in the main report's Data Visualization section.

```
# Create figures directory if it does not exist
dir.create("../reports/figures", recursive = TRUE, showWarnings = FALSE)
```

Figure 1: Risk-Adjusted Daily Return by Asset

Bar chart showing the ratio of mean daily return to standard deviation for each asset. The dashed reference line marks SPY's ratio.

```
ggplot(q1, aes(x = Risk_Adjusted_Return, y = reorder(Asset,
 ↪  Risk_Adjusted_Return),
                fill = Asset)) +
  geom_col(width = 0.6, show.legend = FALSE) +
  geom_vline(xintercept = spy_ratio, linetype = "dashed",
             color = "#B3B3B3", linewidth = 0.4) +
  geom_text(aes(x = Risk_Adjusted_Return / 2,
                label = sprintf("%.4f", Risk_Adjusted_Return)),
            hjust = 0.5, size = 3.2, color = "white", fontface = "bold") +
  annotate("text", x = spy_ratio, y = 4.5, label = "SPY benchmark",
           size = 2.8, color = "#595959", hjust = -0.1) +
  scale_fill_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
                               BTC = "#F7931A", SPY = "#00733E")) +
  labs(title = "Risk-Adjusted Daily Return by Asset",
       subtitle = "Mean return divided by standard deviation, 2020 to 2025",
       x = "Risk-Adjusted Return", y = NULL) +
  xlim(0, 0.07) +
  theme_minimal(base_size = 11) +
  theme(panel.grid.major.y = element_blank(),
        panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"))
```

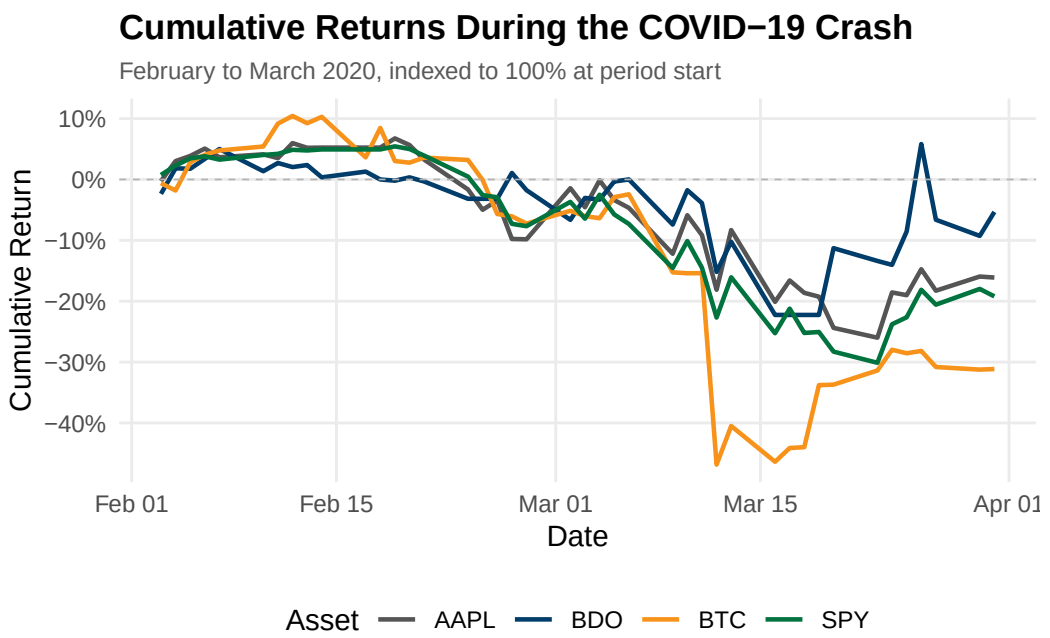



```
ggsave("../reports/figures/fig_q1.pdf", width = 7, height = 3.5, device =
 ↪  "pdf")
```

Figure 2: Cumulative Returns During the COVID-19 Crash

Line chart tracking each asset's cumulative return from February to March 2020, indexed to 100% at period start.

```
ggplot(q2, aes(x = Date, y = Cumulative_Return, color = Asset)) +
  geom_line(linewidth = 0.8) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "#B3B3B3",
   ↪  linewidth = 0.3) +
  scale_color_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
                                BTC = "#F7931A", SPY = "#00733E")) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(title = "Cumulative Returns During the COVID-19 Crash",
       subtitle = "February to March 2020, indexed to 100% at period start",
       x = "Date", y = "Cumulative Return", color = "Asset") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"),
        legend.position = "bottom")
```

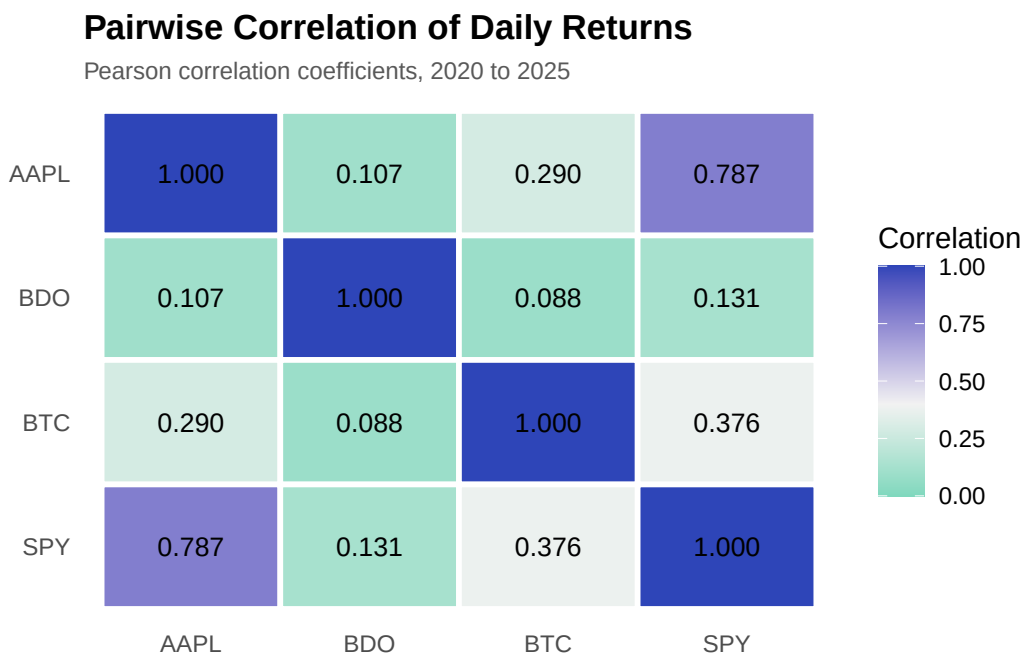



```
ggsave("../reports/figures/fig_q2.pdf", width = 7, height = 4, device =
 ↪  "pdf")
```

Figure 3: Pairwise Correlation Heatmap

Heatmap matrix of Pearson correlation coefficients for daily returns across the four assets.

```
ggplot(cor_long, aes(x = Asset2, y = Asset1, fill = Correlation)) +
  geom_tile(color = "white", linewidth = 1) +
  geom_text(aes(label = sprintf("%.3f", Correlation)), size = 3.5) +
  scale_fill_gradient2(low = "#1DC9A4", mid = "#F2F2F2", high = "#2E45B8",
                       midpoint = 0.4, limits = c(0, 1)) +
  labs(title = "Pairwise Correlation of Daily Returns",
       subtitle = "Pearson correlation coefficients, 2020 to 2025",
       x = NULL, y = NULL, fill = "Correlation") +
  theme_minimal(base_size = 11) +
  theme(panel.grid = element_blank(),
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"),
        legend.position = "right")
```

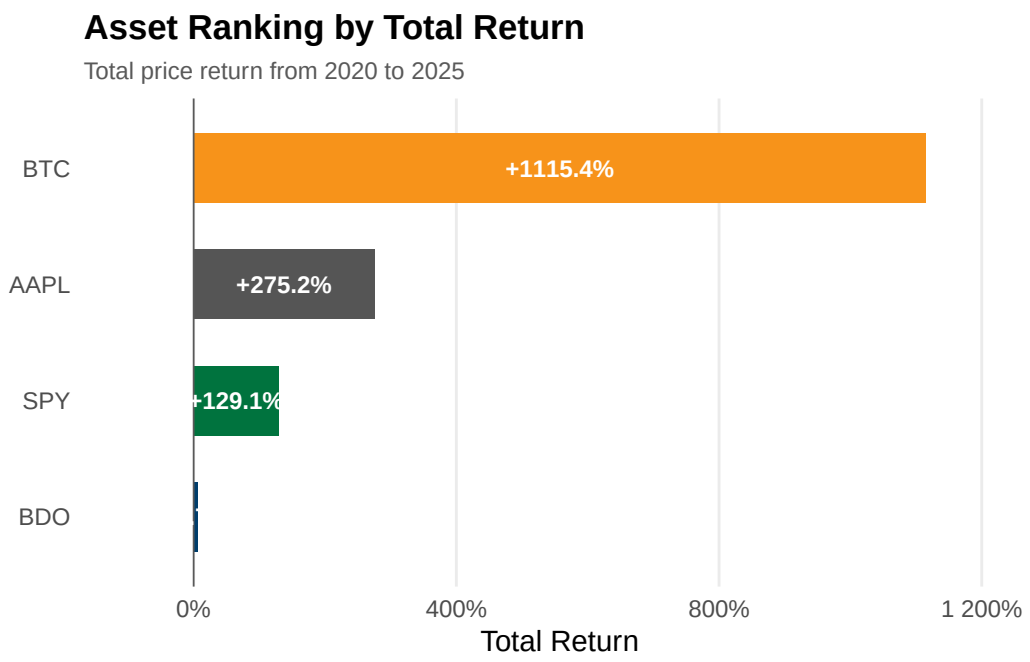



```
ggsave("../reports/figures/fig_q3.pdf", width = 6, height = 4.5, device =
↪  "pdf")
```

Figure 4: Asset Ranking by Total Return

Horizontal bar chart ranking assets by total percentage price return from 2020 to 2025.

```
ggplot(q4_plot, aes(x = Total_Return, y = Asset, fill = Asset)) +
  geom_col(width = 0.6, show.legend = FALSE) +
  geom_vline(xintercept = 0, linewidth = 0.3, color = "#595959") +
  geom_text(aes(x = Total_Return / 2,
                label = sprintf("%+.1f%%", Total_Return * 100)),
            hjust = 0.5, size = 3.2, color = "white", fontface = "bold") +
  scale_fill_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
                               BTC = "#F7931A", SPY = "#00733E")) +
  scale_x_continuous(labels = scales::percent_format(),
                     expand = expansion(mult = c(0.15, 0.15))) +
  labs(title = "Asset Ranking by Total Return",
       subtitle = "Total price return from 2020 to 2025",
       x = "Total Return", y = NULL) +
  theme_minimal(base_size = 11) +
  theme(panel.grid.major.y = element_blank(),
        panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"))
```



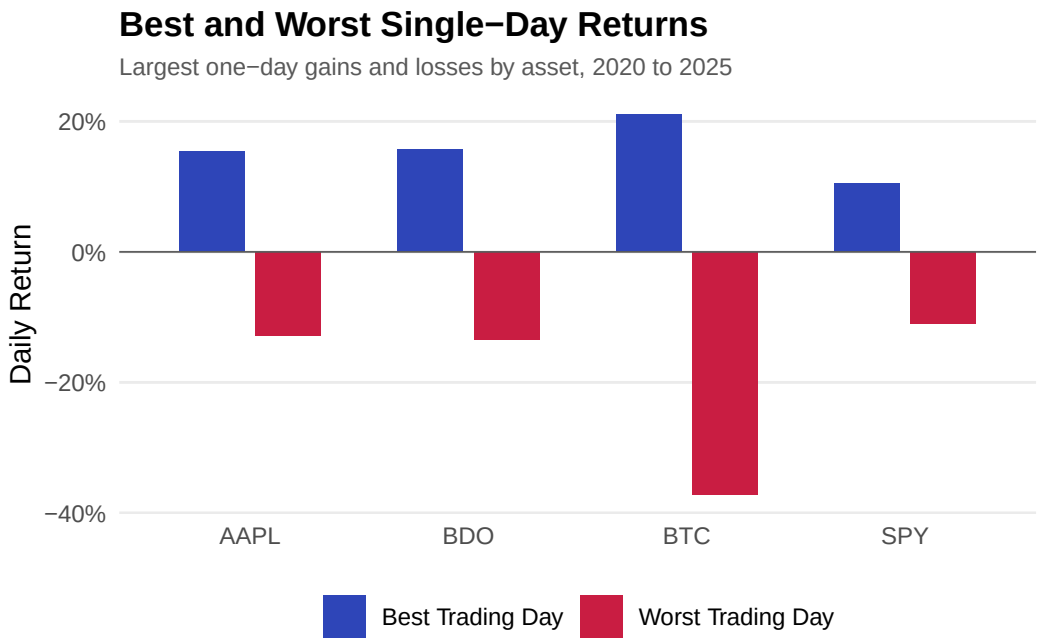

```
ggsave("../reports/figures/fig_q4.pdf", width = 7, height = 3.5, device =
↪  "pdf")
```

Figure 5: Best and Worst Single-Day Returns

Grouped bar chart comparing the largest single-day gain and loss for each asset.

```
q5_plot <- q5 |>
  select(Asset, Best_Daily_Return, Worst_Daily_Return) |>
  pivot_longer(cols = c(Best_Daily_Return, Worst_Daily_Return),
               names_to = "Trading_Day_Type",
               values_to = "Daily_Return") |>
  mutate(
    Trading_Day_Type = recode(Trading_Day_Type,
                              Best_Daily_Return = "Best Trading Day",
                              Worst_Daily_Return = "Worst Trading Day")
  )

ggplot(q5_plot, aes(x = Asset, y = Daily_Return, fill = Trading_Day_Type)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  geom_hline(yintercept = 0, linewidth = 0.3, color = "#595959") +
  scale_fill_manual(values = c("Best Trading Day" = "#2E45B8",
                               "Worst Trading Day" = "#C91D42")) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(title = "Best and Worst Single-Day Returns",
       subtitle = "Largest one-day gains and losses by asset, 2020 to 2025",
       x = NULL, y = "Daily Return", fill = NULL) +
  theme_minimal(base_size = 11) +
```

```
  theme(panel.grid.major.x = element_blank(),
        panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"),
        legend.position = "bottom")
```



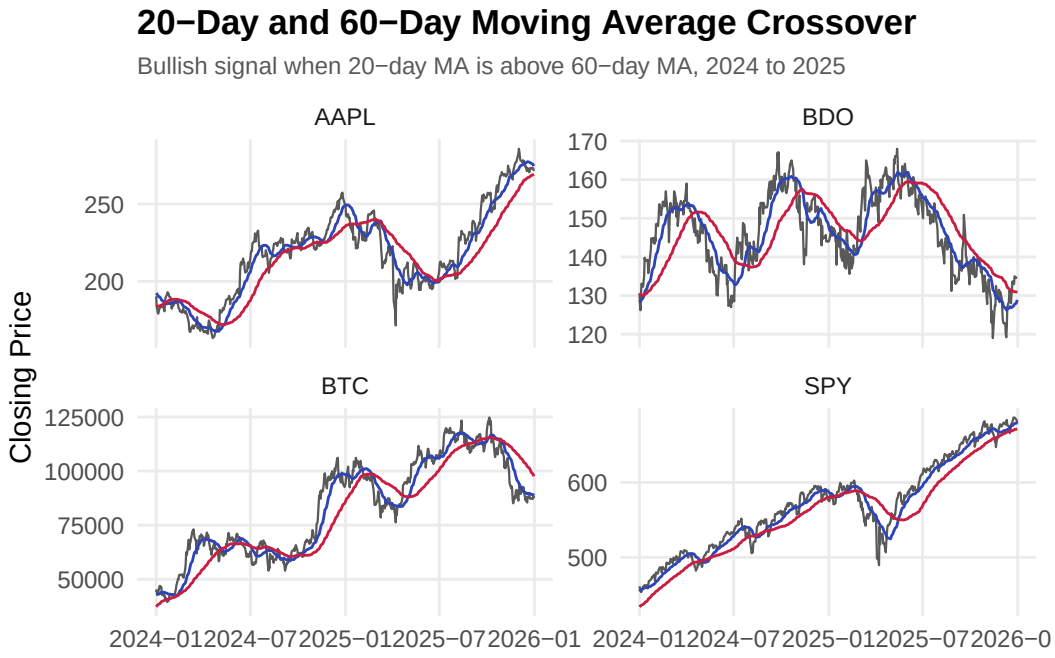

```
ggsave("../reports/figures/fig_q5.pdf", width = 7, height = 4, device =
 ↪  "pdf")
```

Figure 6: 20-Day and 60-Day Moving Average Crossover

Multi-panel faceted line chart showing closing prices with 20-day and 60-day moving averages from 2024 to 2025.

```
q6_plot <- q6_ma |>
  filter(Date >= as.Date("2024-01-01"))

ggplot(q6_plot, aes(x = Date)) +
  geom_line(aes(y = Close), linewidth = 0.4, color = "#595959") +
  geom_line(aes(y = MA_20), linewidth = 0.5, color = "#2E45B8") +
  geom_line(aes(y = MA_60), linewidth = 0.5, color = "#C91D42") +
  facet_wrap(~ Asset, scales = "free_y") +
  labs(title = "20-Day and 60-Day Moving Average Crossover",
       subtitle = "Bullish signal when 20-day MA is above 60-day MA, 2024 to
        ↪  2025",
       x = NULL, y = "Closing Price") +
  theme_minimal(base_size = 11) +
```

```
  theme(panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"))
```



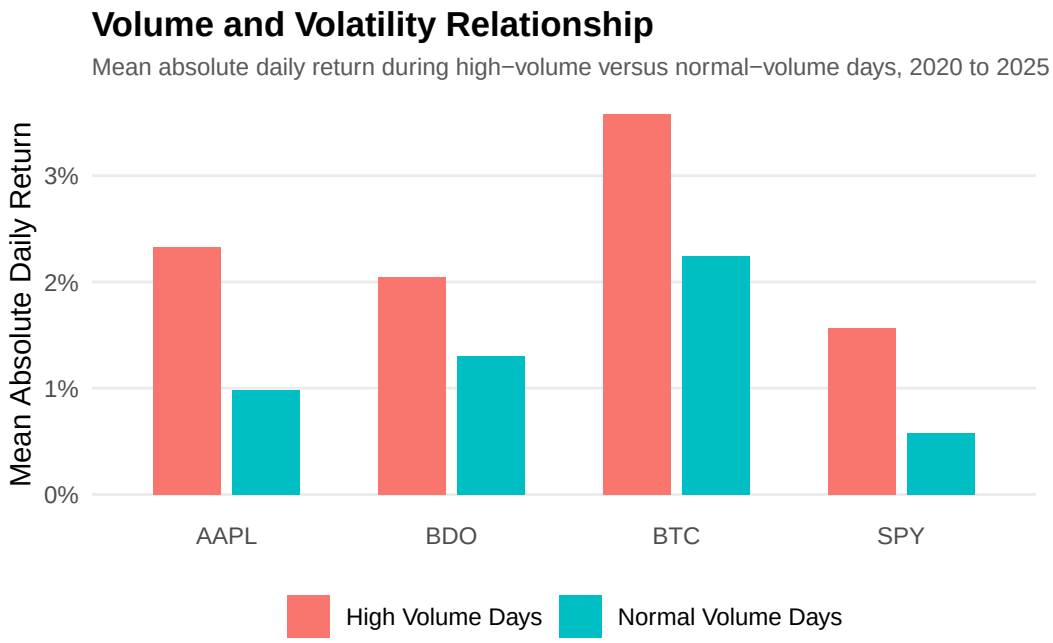

```
ggsave("../reports/figures/fig_q6.pdf", width = 7, height = 4.5, device =
 ↪  "pdf")
```

Figure 7: Volume and Volatility Relationship

Grouped bar chart comparing mean absolute daily return during high-volume versus normal-volume days.

```
ggplot(q7, aes(x = Asset, y = Mean_Absolute_Return, fill = Volume_Regime)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(
    title = "Volume and Volatility Relationship",
    subtitle = "Mean absolute daily return during high-volume versus
     ↪  normal-volume days, 2020 to 2025",
    x = NULL,
    y = "Mean Absolute Daily Return",
    fill = NULL
  ) +
  theme_minimal(base_size = 11) +
  theme(
    panel.grid.major.x = element_blank(),
```

```
    panel.grid.minor = element_blank(),
    axis.ticks = element_blank(),
    plot.title = element_text(face = "bold", size = 13),
    plot.subtitle = element_text(size = 9, color = "#595959"),
    legend.position = "bottom"
  )
```



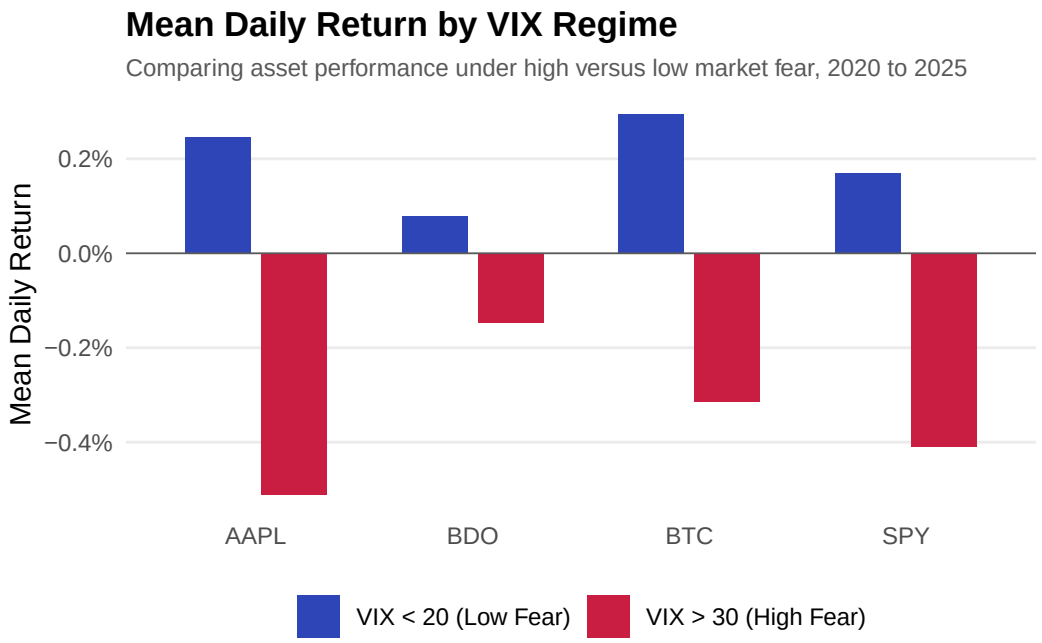

```
ggsave("../reports/figures/fig_q7.pdf", width = 7, height = 4, device =
 ↪  "pdf")
```

Figure 8: Mean Daily Return by VIX Regime

Grouped bar chart comparing asset returns during high-fear (VIX $>$ 30) versus low-fear (VIX $<$ 20) regimes.

```
ggplot(q8_plot, aes(x = Asset, y = Mean_Daily_Return, fill = VIX_Regime)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  geom_hline(yintercept = 0, linewidth = 0.3, color = "#595959") +
  scale_fill_manual(values = c("VIX > 30 (High Fear)" = "#C91D42",
                               "VIX < 20 (Low Fear)" = "#2E45B8")) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(title = "Mean Daily Return by VIX Regime",
       subtitle = "Comparing asset performance under high versus low market
        ↪  fear, 2020 to 2025",
       x = NULL, y = "Mean Daily Return", fill = NULL) +
  theme_minimal(base_size = 11) +
  theme(panel.grid.major.x = element_blank(),
        panel.grid.minor = element_blank(),
```

```
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"),
        legend.position = "bottom")
```



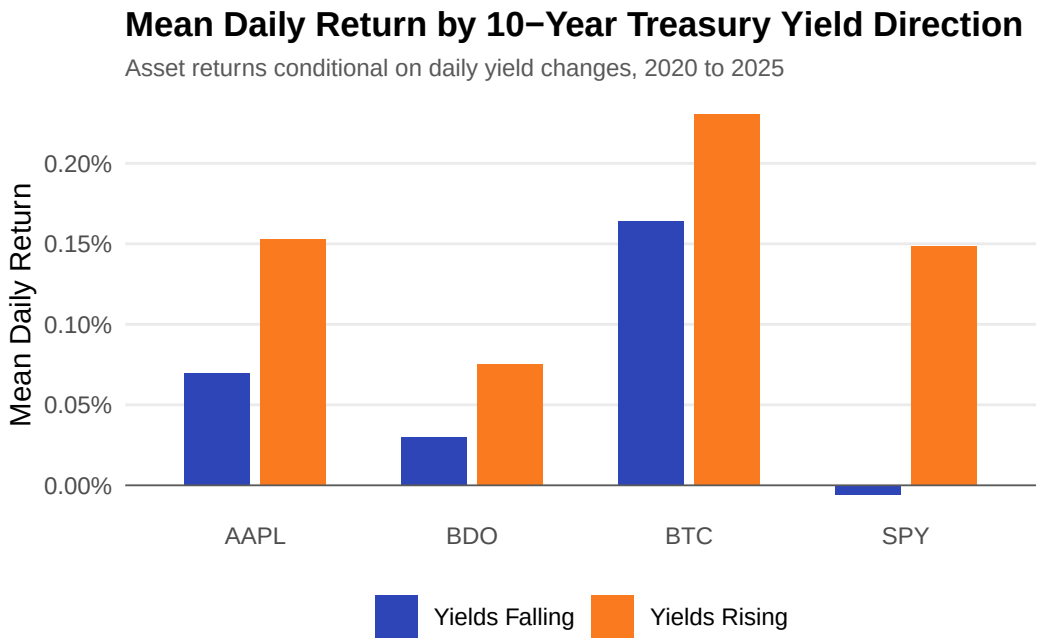

```
ggsave("../reports/figures/fig_q8.pdf", width = 7, height = 4, device =
↪  "pdf")
```

Figure 9: Mean Daily Return by 10-Year Treasury Yield Direction

Grouped bar chart showing asset returns conditional on whether daily yields are rising or falling.

```
ggplot(q9_plot, aes(x = Asset, y = Mean_Daily_Return, fill =
↪  Yield_Direction)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  geom_hline(yintercept = 0, linewidth = 0.3, color = "#595959") +
  scale_fill_manual(values = c("Yields Rising" = "#F97A1F",
                               "Yields Falling" = "#2E45B8")) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(title = "Mean Daily Return by 10-Year Treasury Yield Direction",
       subtitle = "Asset returns conditional on daily yield changes, 2020 to
       ↪  2025",
       x = NULL, y = "Mean Daily Return", fill = NULL) +
  theme_minimal(base_size = 11) +
  theme(panel.grid.major.x = element_blank(),
        panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
```

```
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"),
        legend.position = "bottom")
```



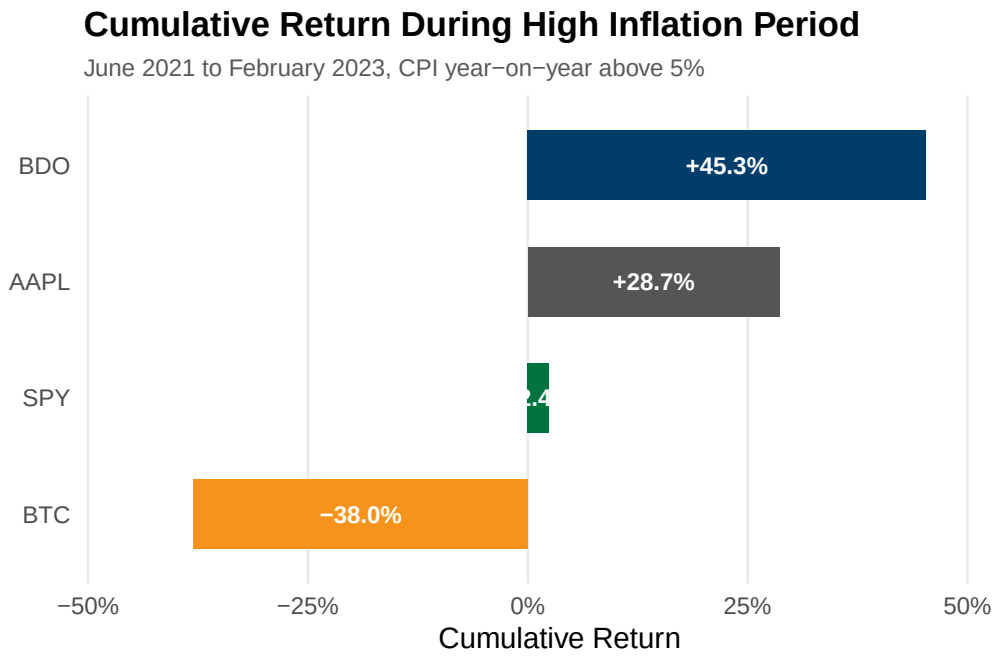

```
ggsave("../reports/figures/fig_q9.pdf", width = 7, height = 4, device =
 ↪  "pdf")
```

Figure 10: Cumulative Return During High Inflation Period

Horizontal bar chart showing total cumulative returns during the period when CPI year-on-year exceeded 5%.

```
ggplot(q10_plot, aes(x = Total_Cumulative_Return, y = Asset, fill = Asset))
 ↪  +
  geom_col(width = 0.6, show.legend = FALSE) +
  geom_text(aes(x = Total_Cumulative_Return / 2,
                label = sprintf("%+.1f%%", Total_Cumulative_Return * 100)),
            hjust = 0.5, size = 3.2, color = "white", fontface = "bold") +
  scale_fill_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
                               BTC = "#F7931A", SPY = "#00733E")) +
  scale_x_continuous(labels = scales::percent_format(),
                     expand = expansion(mult = c(0.15, 0.15))) +
  labs(title = "Cumulative Return During High Inflation Period",
       subtitle = "June 2021 to February 2023, CPI year-on-year above 5%",
       x = "Cumulative Return", y = NULL) +
  theme_minimal(base_size = 11) +
  theme(panel.grid.major.y = element_blank(),
        panel.grid.minor = element_blank(),
```

```
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"))
```




```
ggsave("../reports/figures/fig_q10.pdf", width = 7, height = 3.5, device =
↪  "pdf")
```

Appendix A19 - Exploratory Data Visualization

Author: SEBALLOS, Josiah Dweyn Panganiban Contains 8 exploratory figures covering cumulative returns, price trends, return distributions, rolling volatility, drawdown analysis, risk-return profiles, and portfolio risk contribution. These complement the SQL-driven charts in A17.

Viz Figure 1: Cumulative Returns by Asset

Tracks the growth of USD 1 invested in each asset from 2020 to 2025, indexed to 100% at the start.

```
fig1_data <- sql_long |>
  filter(!is.na(Daily_Return)) |>
  group_by(Asset) |>
  arrange(Date) |>
  mutate(Cumulative_Return = cumprod(1 + Daily_Return) - 1) |>
  ungroup()

ggplot(fig1_data, aes(x = Date, y = Cumulative_Return, color = Asset)) +
```

```
  geom_line(linewidth = 0.8) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "#B3B3B3",
   ↪  linewidth = 0.3) +
  scale_color_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
                                BTC = "#F7931A", SPY = "#00733E")) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(title = "Cumulative Returns by Asset",
       subtitle = "Growth of $1 invested, 2020 to 2025",
       x = "Date", y = "Cumulative Return", color = "Asset") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"),
        legend.position = "bottom")
```

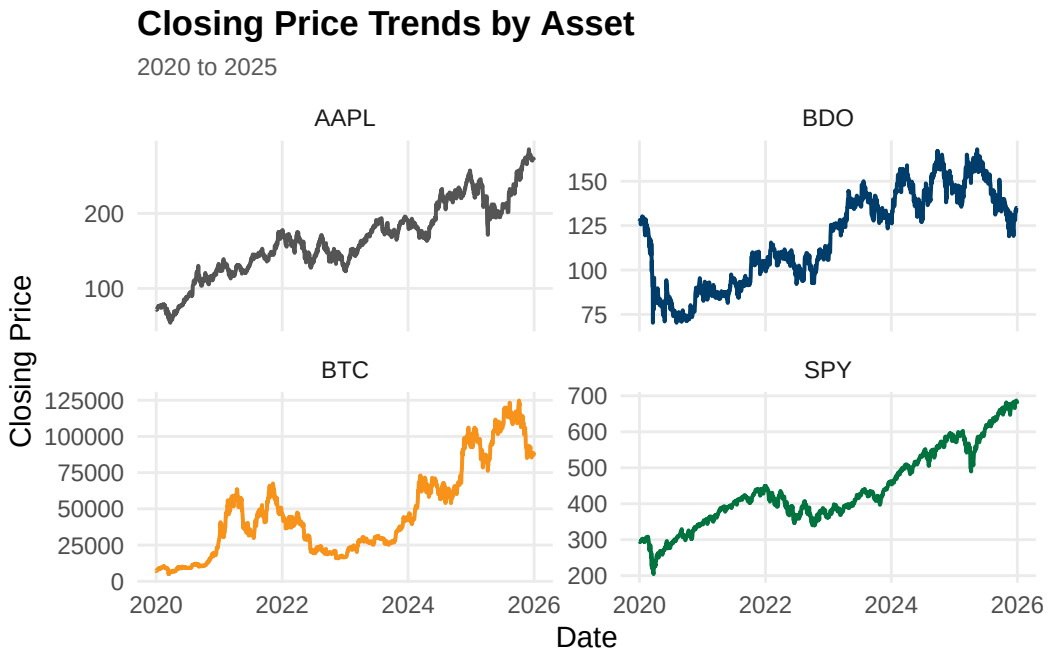

```
ggsave("../reports/figures/fig_viz1.pdf", width = 7, height = 4, device =
 ↪  "pdf")
```

Viz Figure 2: Closing Price Trends by Asset

Plots nominal closing prices on individual scales to compare price-level dynamics across assets of different magnitudes.

```
ggplot(sql_long, aes(x = Date, y = Close, color = Asset)) +
  geom_line(linewidth = 0.7, show.legend = FALSE, na.rm = TRUE) +
  facet_wrap(~ Asset, scales = "free_y", ncol = 2) +
  scale_color_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
```

```
                                  BTC = "#F7931A", SPY = "#00733E")) +
  labs(title = "Closing Price Trends by Asset",
       subtitle = "2020 to 2025",
       x = "Date", y = "Closing Price") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"))
```



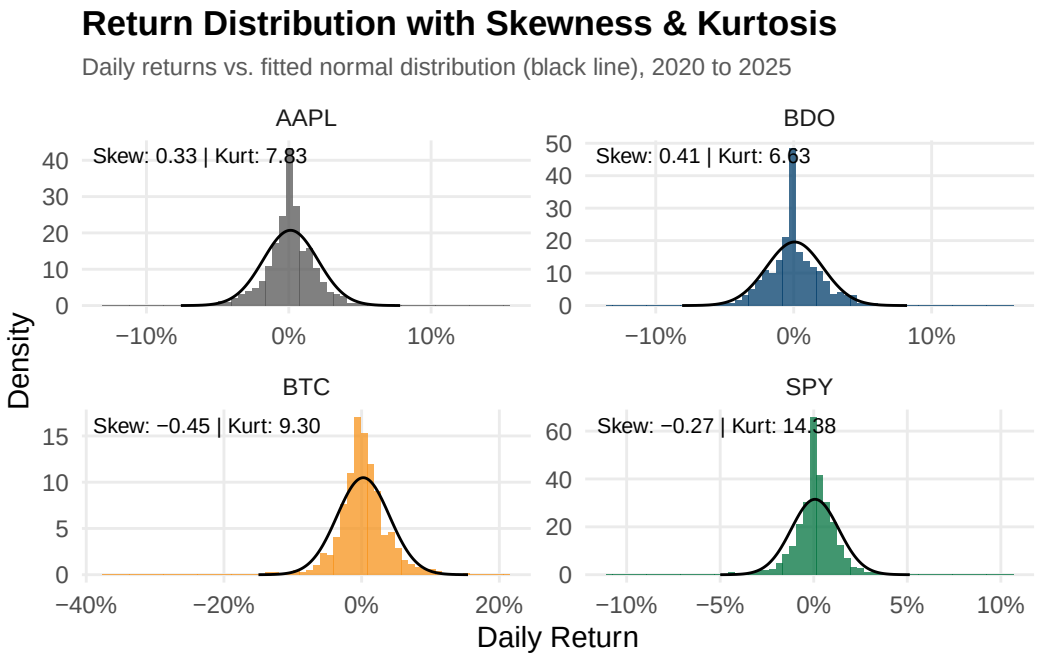

```
ggsave("../reports/figures/fig_viz2.pdf", width = 7, height = 5, device =
 ↪  "pdf")
```

Viz Figure 3: Return Distribution with Skewness and Kurtosis

Overlays each asset's daily return histogram against a fitted normal curve to assess distribution shape and tail risk.

```
fig3_data <- sql_long |> filter(!is.na(Daily_Return))

calc_skew <- function(x) {
  n <- length(x); m <- mean(x)
  (sum((x - m)^3) / n) / (sd(x)^3)
}
calc_kurt <- function(x) {
  n <- length(x); m <- mean(x)
  (sum((x - m)^4) / n) / (sd(x)^2)^2 - 3
```

```
}

fig3_stats <- fig3_data |>
  group_by(Asset) |>
  summarise(
    Mean = mean(Daily_Return), SD = sd(Daily_Return),
    Skewness = calc_skew(Daily_Return), Kurtosis = calc_kurt(Daily_Return),
    .groups = "drop"
  ) |>
  mutate(label = sprintf("Skew: %.2f | Kurt: %.2f", Skewness, Kurtosis))

normal_curves <- fig3_stats |>
  rowwise() |>
  reframe(
    Asset = Asset,
    x = seq(Mean - 4 * SD, Mean + 4 * SD, length.out = 200),
    y = dnorm(x, mean = Mean, sd = SD)
  )

ggplot(fig3_data, aes(x = Daily_Return)) +
  geom_histogram(aes(y = after_stat(density), fill = Asset), bins = 60,
                 alpha = 0.75, show.legend = FALSE) +
  geom_line(data = normal_curves, aes(x = x, y = y), color = "black",
   ↪  linewidth = 0.5) +
  geom_text(data = fig3_stats, aes(x = -Inf, y = Inf, label = label),
            hjust = -0.05, vjust = 1.5, size = 2.8) +
  facet_wrap(~ Asset, scales = "free", ncol = 2) +
  scale_fill_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
                               BTC = "#F7931A", SPY = "#00733E")) +
  scale_x_continuous(labels = scales::percent_format()) +
  labs(title = "Return Distribution with Skewness & Kurtosis",
       subtitle = "Daily returns vs. fitted normal distribution (black
        ↪  line), 2020 to 2025",
       x = "Daily Return", y = "Density") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"))
```

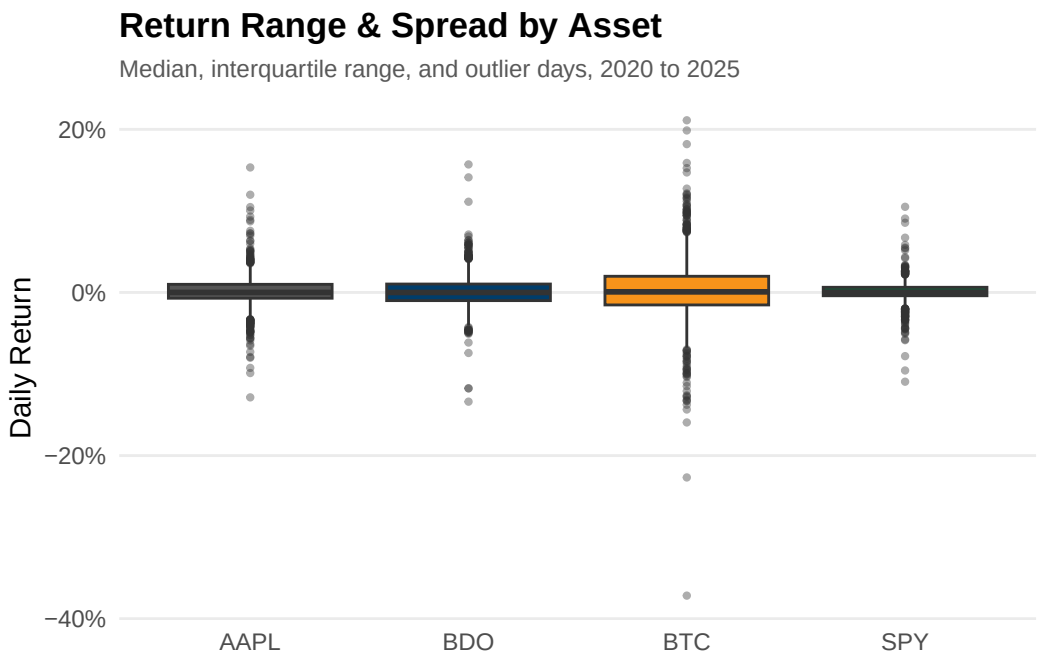



```
ggsave("../reports/figures/fig_viz3.pdf", width = 7, height = 5, device =
 ↪  "pdf")
```

Viz Figure 4: Return Range and Spread by Asset

Boxplot showing median, interquartile range, and outlier days for each asset.

```
ggplot(fig3_data, aes(x = Asset, y = Daily_Return, fill = Asset)) +
  geom_boxplot(show.legend = FALSE, outlier.size = 0.8, outlier.alpha = 0.4)
   ↪  +
  scale_fill_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
                               BTC = "#F7931A", SPY = "#00733E")) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(title = "Return Range & Spread by Asset",
       subtitle = "Median, interquartile range, and outlier days, 2020 to
        ↪  2025",
       x = NULL, y = "Daily Return") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.major.x = element_blank(),
        panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"))
```



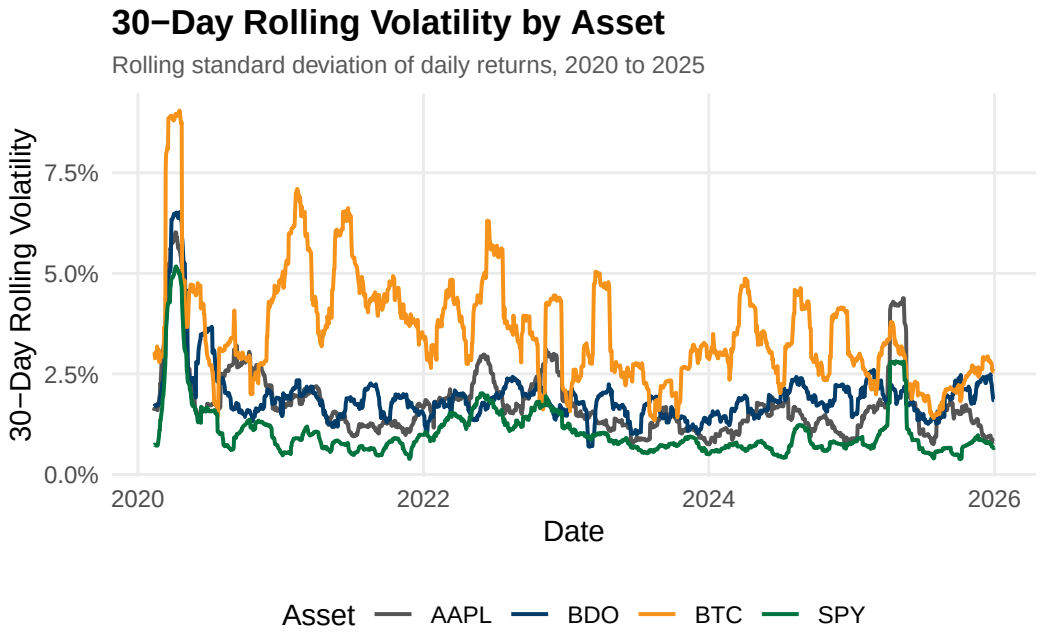

```
ggsave("../reports/figures/fig_viz4.pdf", width = 7, height = 4, device =
↪  "pdf")
```

Viz Figure 5: 30-Day Rolling Volatility by Asset

Tracks rolling 30-day standard deviation of daily returns to visualize volatility evolution over time.

```
rolling_sd <- function(x, n = 30) {
  sapply(seq_along(x), function(i) {
    if (i < n) return(NA)
    sd(x[(i - n + 1):i], na.rm = TRUE)
  })
}

fig5_data <- sql_long |>
  filter(!is.na(Daily_Return)) |>
  group_by(Asset) |>
  arrange(Date) |>
  mutate(Rolling_Vol = rolling_sd(Daily_Return, 30)) |>
  ungroup() |>
  filter(!is.na(Rolling_Vol))

ggplot(fig5_data, aes(x = Date, y = Rolling_Vol, color = Asset)) +
  geom_line(linewidth = 0.7) +
  scale_color_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
                                BTC = "#F7931A", SPY = "#00733E")) +
  scale_y_continuous(labels = scales::percent_format()) +
```

```
  labs(title = "30-Day Rolling Volatility by Asset",
       subtitle = "Rolling standard deviation of daily returns, 2020 to
        ↪  2025",
       x = "Date", y = "30-Day Rolling Volatility", color = "Asset") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"),
        legend.position = "bottom")
```



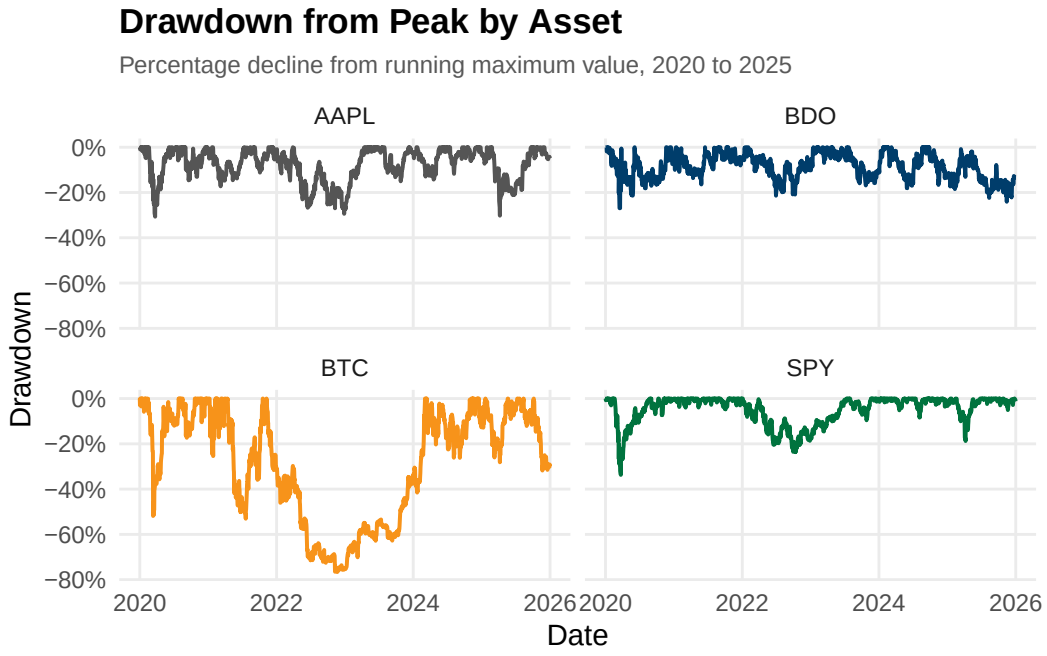

```
ggsave("../reports/figures/fig_viz5.pdf", width = 7, height = 4, device =
 ↪  "pdf")
```

Viz Figure 6: Drawdown from Peak by Asset

Computes percentage decline from each asset's running historical high to visualize recovery patterns.

```
fig6_data <- fig1_data |>
  group_by(Asset) |>
  arrange(Date) |>
  mutate(Running_Max = cummax(1 + Cumulative_Return),
         Drawdown = (1 + Cumulative_Return) / Running_Max - 1) |>
  ungroup()

ggplot(fig6_data, aes(x = Date, y = Drawdown, color = Asset)) +
  geom_line(linewidth = 0.7) +
```

```
  facet_wrap(~ Asset, ncol = 2) +
  scale_color_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
                                BTC = "#F7931A", SPY = "#00733E")) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(title = "Drawdown from Peak by Asset",
       subtitle = "Percentage decline from running maximum value, 2020 to
        ↪  2025",
       x = "Date", y = "Drawdown") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        legend.position = "none",
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"))
```



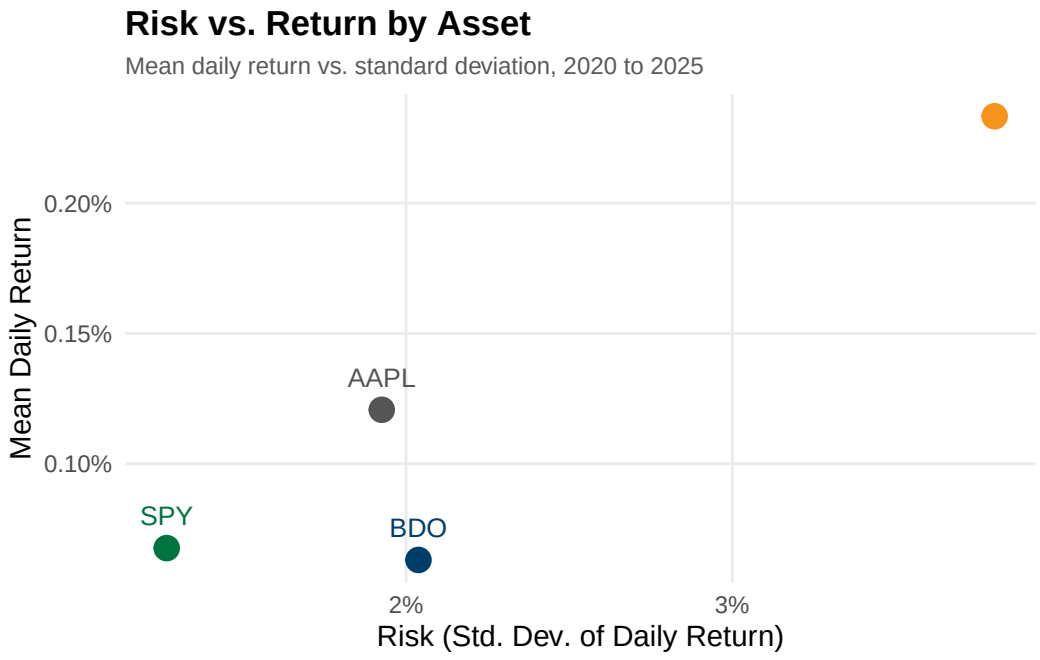

```
ggsave("../reports/figures/fig_viz6.pdf", width = 7, height = 5, device =
 ↪  "pdf")
```

Viz Figure 7: Risk vs. Return by Asset

Scatter plot comparing mean daily return against standard deviation across all four assets.

```
fig7_data <- sql_long |>
  filter(!is.na(Daily_Return)) |>
  group_by(Asset) |>
  summarise(Mean_Return = mean(Daily_Return), SD_Return = sd(Daily_Return),
   ↪  .groups = "drop")
```

```
ggplot(fig7_data, aes(x = SD_Return, y = Mean_Return, color = Asset)) +
  geom_point(size = 4) +
  geom_text(aes(label = Asset), vjust = -1.2, size = 3.5, show.legend =
  ↪  FALSE) +
  scale_color_manual(values = c(AAPL = "#555555", BDO = "#003D6B",
                                BTC = "#F7931A", SPY = "#00733E")) +
  scale_x_continuous(labels = scales::percent_format()) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(title = "Risk vs. Return by Asset",
       subtitle = "Mean daily return vs. standard deviation, 2020 to 2025",
       x = "Risk (Std. Dev. of Daily Return)", y = "Mean Daily Return") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        legend.position = "none",
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"))
```



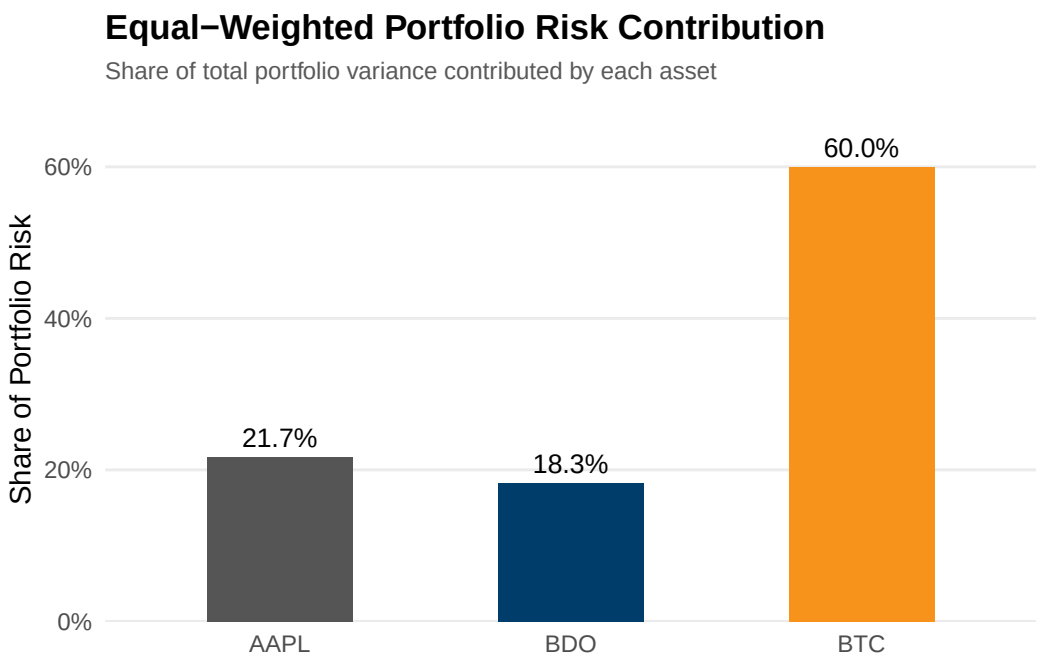

```
ggsave("../reports/figures/fig_viz7.pdf", width = 7, height = 5, device =
 ↪  "pdf")
```

Viz Figure 8: Equal-Weighted Portfolio Risk Contribution

Decomposes total portfolio variance into each asset's contribution to show that equal dollar weights do not produce equal risk. SPY is excluded because Apple's 0.787 correlation with the S&P 500 means that including both AAPL and SPY would double-count the same equity

factor exposure, inflating its risk contribution.

```
returns_matrix <- sql_long |>
  filter(Asset %in% c("AAPL", "BDO", "BTC"), !is.na(Daily_Return)) |>
  select(Date, Asset, Daily_Return) |>
  pivot_wider(names_from = Asset, values_from = Daily_Return) |>
  select(AAPL, BDO, BTC) |>
  na.omit()

cov_matrix <- cov(returns_matrix)
weights <- c(AAPL = 1/3, BDO = 1/3, BTC = 1/3)
port_var <- as.numeric(t(weights) %*% cov_matrix %*% weights)
marginal_contrib <- cov_matrix %*% weights
risk_contrib <- as.numeric(weights * marginal_contrib / port_var)

fig8_data <- data.frame(Asset = names(weights), Risk_Contribution =
 ↪  risk_contrib)

ggplot(fig8_data, aes(x = Asset, y = Risk_Contribution, fill = Asset)) +
  geom_col(width = 0.5, show.legend = FALSE) +
  geom_text(aes(label = scales::percent(Risk_Contribution, accuracy = 0.1)),
            vjust = -0.5, size = 3.5) +
  scale_fill_manual(values = c(AAPL = "#555555", BDO = "#003D6B", BTC =
   ↪  "#F7931A")) +
  scale_y_continuous(labels = scales::percent_format(), expand =
   ↪  expansion(mult = c(0, 0.15))) +
  labs(title = "Equal-Weighted Portfolio Risk Contribution",
       subtitle = "Share of total portfolio variance contributed by each
        ↪  asset",
       x = NULL, y = "Share of Portfolio Risk") +
  theme_minimal(base_size = 11) +
  theme(panel.grid.major.x = element_blank(),
        panel.grid.minor = element_blank(),
        axis.ticks = element_blank(),
        plot.title = element_text(face = "bold", size = 13),
        plot.subtitle = element_text(size = 9, color = "#595959"))
```




```
ggsave("../reports/figures/fig_viz8.pdf", width = 7, height = 4, device =
↪  "pdf")
```

Appendix A20 - Investment Recommendation (Excel Solver)

Note: Asset allocation was optimized using Excel Solver, minimizing portfolio variance subject to return and diversification constraints.